Flaris Language Guide

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.6.0 **Generated** 2026-10-24 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Table of Contents

1. Introduction
2. Getting Started - CLI
3. Lexical Structure
4. Variables & Types
5. Expressions
6. Control Flow
7. Functions
8. Classes
9. Fibers & Async
10. Working with JSON
11. Analyzer & Diagnostics
12. Debugging
13. 10-Minute Tours
14. Package Manager (flarispm)

---

1. Introduction

Flaris is a lightweight, embedded scripting language designed for high performance, portability, and deep integration with host applications. It combines the expressiveness of modern scripting languages with the control of a virtual machine built around **fibers**, **async/await**, and **cooperative scheduling**.

Flaris is:

- **Deterministic** - execution order is always clear and reproducible.
- **Fast** - written in C with a compact bytecode interpreter.
- **Embeddable** - ideal for applications, tools, servers, and games.
- **Safe by default** - unsafe features (FFI, raw pointers/memory) require explicit enabling (`--unsafe`, or `--allow-ffi` for FFI alone). Static analysis runs on every compile and catches mistakes before the program starts. A runtime-only binary (`flaris`) ships without the compiler, `VM.Eval` or `VM.Compile`.
- **Concurrent** - fibers and async functions allow concurrency without threads.

Three core principles:

1. **Explicit over implicit** - nothing runs unless you call or resume it.
2. **Minimal magic** - the runtime is predictable, transparent, and debuggable.
3. **Simplicity without compromise** - features are easy to use but designed to scale.

Typical use cases: game scripting, tool automation, embedded device logic, interactive consoles, server-side utilities, scripting in C/C++ software, simulation and testing frameworks.

Supported Targets

Flaris targets 64-bit platforms: x86-64 and ARM64 on Linux, macOS, and Windows. Compiled `.flx` bytecode is portable between all of them - you can compile on Apple-silicon macOS and deploy on an x86-64 Linux server without recompiling.

The one exception is code that works with raw memory addresses (FFI, `Memory.*`, `Buffer.GetAddress`). Addresses are process-local and should never be serialized or shared across machines. Use `Os.Arch()` and `Os.Name()` to guard any platform-specific paths.

---

2. Getting Started - CLI

The `flarisvm` executable is how you run Flaris programs.

Invocation Modes

Compile and run (most common):

```
flarisvm script.fls
```

Scripts. A file that starts with `#!` (a shebang line) is a script: its top-level statements run as the body of a synthesized `Main()`, so there is no `Main` to write. `fn`, `class` and `import` declarations may appear anywhere in the file, before or after statements; a `const`, `enum`, `global` or `export` that follows the first statement belongs to `Main`.

```
#!/usr/bin/env flarisvm
Console.WriteLine(greet("world"));
fn greet(name) { return "hello, " + name; }
```

Compile to bytecode (for distribution):

```
flarisvm --compile script.fls output.flx
```

For performance:

```
flarisvm --compile script.fls output.flx --strip --small
```

Run precompiled bytecode:

```
flarisvm --exec script.flx
```

Read source from stdin - use `-` as the filename:

```
echo 'fn Main() { Console.WriteLine("hi"); }' | flarisvm -
cat script.fls | flarisvm - --verbose
```

Compile from stdin to bytecode:

```
cat script.fls | flarisvm --compile - output.flx
```

Execute bytecode from stdin:

```
cat output.flx | flarisvm --exec -
```

The stdin sentinel `-` works wherever a source or bytecode filename is accepted. This enables pipelines where the program never touches the filesystem:

```
generate_code | flarisvm -       # generate and run on the fly
```

Disassemble bytecode (debug):

```
flarisvm --disasm script.flx
```

Check JIT eligibility - compile without running and print, per function, whether it runs as native code and (if not) why:

```
flarisvm --check script.fls
# ✓ dot          (line 20)
# · label        (line 34) - operator on a non-numeric operand (e.g. string concatenation)
```

Every eligible function is JIT-compiled automatically (no annotation needed; `--jit-disable` turns it off) - `--check` shows you which ones qualify. See Reference R11.

Run inline code (quick test):

```
flarisvm --eval 'Console.WriteLine("Hello");'
```

Get SHA-256 fingerprint (for imports):

```
flarisvm --sha256 script.flx
```

Self-contained binaries

Bundle a program and the runtime into a single native executable - one file to distribute, with nothing to install on the target machine (similar to Go). `--embed` compiles your program (or takes precompiled `.flx`) and appends it to the small compiler-free `flaris` runtime; the result runs your program directly when launched.

From source, in one step:

```
flarisvm --embed app.fls myapp
./myapp arg1 arg2          # runs standalone - no flaris on PATH, no --libs
```

Statically link library dependencies with `--bundle`, so the shipped binary needs no `~/.flaris/libs`. On its own it resolves the program's whole import chain and packs exactly that set; with an explicit list it packs the files you name:

```
flarisvm --embed app.fls myapp --bundle                # every library the program imports
flarisvm --embed app.fls myapp --bundle='./mylib.flx'  # exactly these files
```

Embed precompiled bytecode directly (detected by content, not extension):

```
flarisvm --embed app.flx myapp
```

Bytecode is embedded byte for byte, which keeps its signature and recorded identity intact - so `--bundle` cannot apply to a `.flx` input and is refused rather than ignored. Link the dependencies when compiling and embed the bundle that produces:

```
flarisvm --compile app.fls app.flx --bundle
flarisvm --embed app.flx myapp
```

`--embed <input> <output>` accepts the same `--bundle`, `--sign`, `--no-opt`, and `--strip` options as `--compile`. Runtime behavior is also decided here: pass `--jit-disable`, `--unsafe`, or VM limits (`--stack=`, `--frames=`, ...) alongside `--embed` and they are baked into the binary and applied on every start - a standalone binary hands all its CLI arguments to the script, so embedding is the only place to set them (nothing is enabled by default). It needs the `flaris` runtime stub next to `flarisvm` or on `PATH` (the official installer places both). The stub has no compiler and no `eval`, so a bundled binary ships only the bytecode you built. Runs on Linux, macOS (including Apple Silicon), and Windows - the payload rides in trailing space every executable format tolerates; see **Reference R1** for platform and signing notes.

Package signing (Ed25519)

`.flx` bytecode can carry an embedded Ed25519 signature. The official standard library is signed with the `flaris-lang.org` key, which is compiled into the runtime as a built-in trust anchor - so signed code verifies with nothing to set up. A tampered file always fails to load; `--require-signed` additionally refuses anything unsigned or signed by an untrusted key. (Full details in `reference.md`.)

Sign when compiling (the secret key inline or as a keyfile path):

```
flarisvm --compile mylib.fls mylib.flx --sign=publisher.sk
```

Inspect a file's signature (machine-readable - used by flarispm):

```
flarisvm --sig-info mylib.flx
# unsigned
# signed|<trusted|valid|invalid>|<pubkey-hex>|<signer>
```

Trust a publisher (append a key to `~/.flaris/trusted_keys`):

```
flarisvm --import-key "acme corp" <ed25519-pubkey-hex>
```

Enforce signatures at load:

```
flarisvm --require-signed --exec app.flx
```

Common Flags

Flag	Effect
`--verbose`	Verbose output (timing, info)
`--trace`	Very verbose (opcode-level trace)
`--time`	Timing report
`--jit-disable`	Turn off the JIT. It is on by default and auto-compiles every eligible function to native code (use `--check` to see which)
`--mem`	Memory report at shutdown (peak RSS, slab footprint, leak detection)
`--stats`	Statistics (allocations, counters)
`--prof[=hz]`	Sample the running script `hz` times a second (default 1000) and print where the time went at exit; `flarisvm` only
`--unsafe`	Enable FFI / unsafe code
`--small`	Enable strip of symbols
`-g` / `--no-strip`	Keep debug info - line numbers in stack traces (off by default)
`--strip`	No debug info - the default; overrides the `-g` that `--debug` implies
`--no-opt`	Disable optimizations
`--no-verify`	Disable FFI library SHA-256 checks (not `library()` pins)
`--require-signed`	Refuse to load any `.flx` that is unsigned, tampered, or signed by an untrusted key

Development workflow:

Show idle time (VM is waiting for fibers or other) and total running time.

```
flarisvm app.fls --verbose --time
```

Production:

```
flarisvm --exec app.flx
```

> See **Reference R1** for the complete CLI specification.

---

3. Lexical Structure

Flaris source files are UTF-8 encoded and must not contain NUL bytes. Identifiers use ASCII letters, digits, and underscores: `[A-Za-z_][A-Za-z0-9_]*`.

Keywords

Keyword	Purpose
`fn`	Declare function
`async`	Mark function as asynchronous
`static`	Declare static method in class
`inline`	Suggest function inlining (hint)
`let` / `var`	Declare mutable variable (identical)
`global`	Declare module-level global variable
`const`	Declare compile-time constant
`if` / `else`	Conditional
`for`	C-style loop
`foreach`	Iterate over collection elements
`iter` / `from` / `to`	Range-based integer loop
`while`	While loop
`switch` / `case` / `default`	Multi-branch conditional
`break` / `continue`	Loop control
`return`	Return from function
`yield`	Yield control from fiber
`await`	Wait for async result
`nil` / `true` / `false`	Literals
`class` / `this` / `super` / `new`	Object-oriented
`import` / `export` / `as`	Modules
`try` / `catch` / `finally` / `throw`	Exceptions
`guard`	Assert non-nil (throws if nil)
`weak`	Handle that does not keep its target alive
`enum`	Define enumeration
`breakpoint`	Debug pause
`in`	Membership test / foreach iterator

Keywords as names: any keyword can follow `.` or label an object-literal key (`o.default`, `{ class: 1, default: 2 }`), and a subset - `as`, `default`, `do`, `from`, `global`, `inline`, `iter`, `library`, `static`, `to` - can even name a variable, parameter, function or class field, e.g. `fn Range(from: int, to: int) { return to - from; }`. Exception: at the start of a statement, `iter`, `global`, `do`, `static` and `inline` still begin their own construct, so `iter = 1;` there is not a valid assignment.

Literals

Nil and Booleans:

```
nil
true
false
```

Integers (64-bit signed):

```
42
0xFF
0b1001
1_000_000     // underscores allowed for readability
```

Floats (IEEE-754 double):

```
3.14
1.0e6
-2.5E-3
3.14_15_92
```

Strings:

```
"hello\nworld 🐱"        // single-line with escape sequences
"""line1
line2"""                 // raw: no escapes, verbatim
let json = """
```

```
    { "ok": true }
    """;                // raw multi-line: closer's indentation stripped, no final newline
let n = 42;
$"n = {n}, hex = {n:X4}" // interpolated: holes take String.Format specs
```

Escape sequences: `\n`, `\r`, `\t`, `\b`, `\f`, `\a`, `\v`, `\"`, `\'`, `\\`, `\xNN`, `\uNNNN`, `\u{N..N}`; anything else after a backslash is a compile error, so write `"\\d+"` or `"""\d+"""` for a regex. In `$"..."` write `{{` and `}}` for literal braces and parenthesize a ternary inside a hole. The reference's *Lexical Structure* has the full rules.

Character literals (single Unicode codepoint):

```
'a'
'å'
'🐱'
'\n'
'\u00E5'
```

Comments

```
// single-line comment

/* multi-line
   comment */

/* nested /* block */ comments are supported */
```

Operators

```
// Arithmetic
+  -  *  /  %  ^^        // ^^ is power

// Bitwise
&  |  ^  ~  <<  >>  >>>

// Logical
&&  ||  !
and  or       // aliases for &&  ||

// Comparison
==  !=  <  <=  >  >=  is

// Assignment + compound
=  +=  -=  *=  /=  %=  ^^=
&=  |=  ^=  <<=  >>=  >>>=  ??=
```

```
// Special
??      // null coalescing
=>      // arrow function
```

Punctuation

Semicolons are **required** at the end of every statement. Newlines do not end statements.

```
let x = 10;   // required semicolon
let y = 20;;  // extra semicolons are tolerated
```

---

4. Variables & Types

Declaring Variables

Use `let` or `var` for **local variables** inside functions and blocks (they are identical):

```
let name = "Alice";
var count = 0;
```

Rules:

- Must always have an initializer - `let x;` is illegal, use `let x = nil;`
- Block-scoped - variables are only visible inside their `{ }` block
- Can be reassigned freely
- `let` and `var` are **not valid at module (top) level** - use `global` there

Use `const` for immutable bindings:

```
const PI = 3.1415926535;
const MAX = 128;
```

`const` is deep: neither the binding nor the object it holds can change:

```
const cfg = { port: 8080 };
cfg.port = 9090;   // ❌ compile error 1005 (a const's properties)
cfg = {};          // ❌ compile error 1005 (reassigning the binding)
```

A change that reaches a const object some other way, through a function parameter for example, raises at run time. Use `let` for an object you mutate.

Globals

Use `global` to declare a module-level global, from the top-level code, or from inside a function or nested scope:

```
global shared_int:int = 2;

fn setup() {
    global config = { debug: true };  // visible everywhere in the module
}
```

- `global` always creates (or warns if already exists) a top-level variable
- Access and mutation of globals is slower than locals - prefer locals in hot paths

Types at a Glance

Flaris is dynamically typed. Every value has a type at runtime.

Primitives: `nil`, `bool`, `int`, `float`, `char`, `string`

Containers: `array`, `object`

Structured: `class`, `instance`, `exception`

Execution: `fiber`, `fn`, `module`

Binary: `block`, `pointer`

Type annotations are **optional**. A value the compiler can see is of the wrong type is a compile error; a value whose type is only known when the program runs is checked as it enters an annotated variable, parameter, field or return value. See **Reference R3** for the full type annotation specification.

Quick example:

```
fn add(a: int, b: int): int {
    return a + b;
}
```

Scoping

Variables are block-scoped. Every `{ }` creates a new scope:

```
let x = 1;
{
    let y = 2;
    Console.WriteLine(x, y);  // 1 2
}
// y is not accessible here
```

Inner scopes can **shadow** outer variables:

```
let x = 1;
{
```

```
    let x = 2;              // shadows outer x
    Console.WriteLine(x);   // 2
}
Console.WriteLine(x);       // 1
```

A loop header is a scope of its own, so the loop variable does not outlive the loop and the next loop is free to reuse the name:

```
for (let i = 0; i < 3; i++) { }
for (let i = 0; i < 3; i++) { }   // a second `i`, not a redeclaration
// i is not accessible here
```

The same applies to `iter` and `foreach` variables, and to a `catch` block's error variable.

Globals

Top-level declarations are module globals - visible anywhere in the file and shared across all fibers:

```
global counter = 0;

fn tick() {
    counter += 1;
}
```

Use `global` to explicitly promote a variable to module scope from inside a function:

```
fn init() {
    global appName = "MyApp";
}
```

Enums

`enum` names a group of related integer constants. Members number from `0`, and each unlisted member continues from the one before it:

```
enum Color { Red, Green, Blue }        // 0, 1, 2

enum Status { Ok = 200, Created, NotFound = 404, Gone }
// Ok = 200, Created = 201, NotFound = 404, Gone = 405
```

Members are plain `int` values folded at compile time, so they cost nothing at run time and work anywhere an int does - arithmetic, array indices, `case` labels:

```
enum Level { Debug, Info, Warn, Error }

fn label(l) {
    switch (l) {
```

```
        case Level.Debug: return "DBG";
        case Level.Warn:  return "WRN";
        default:          return "???";
    }
}
```

A `switch` that labels only members of one enum and has no `default` must handle every member; the analyzer warns about the ones it misses (warning 2021).

Values must be integer literals, optionally negative - `A = 1 + 2` and `B = A` `A = -1` are all rejected. Declare an enum at the top level or inside a function, and `export` it like any other symbol.

There is no reverse mapping: `str(Color.Red)` is `"0"`, not `"Red"`. Keep a lookup array or a `switch` if you need names. Reading a member that was never declared yields `nil` rather than raising an error.

See R2 - `enum` declarations for the full rules.

Closures

Flaris supports closures: inner functions capture variables from their enclosing scope by value at the point the inner function is created. Both parameters and body-local variables of the outer function are captured.

Single capture - parameter:

```
fn makeAdder(x) {
    return fn(n) { return x + n; };
}

let add5  = makeAdder(5);
let add10 = makeAdder(10);
Console.WriteLine(add5(3));   // 8
Console.WriteLine(add10(3));  // 13
```

Each call to `makeAdder` produces an independent closure with its own snapshot of `x`.

Multi-capture - parameter + body local:

```
fn makeGreeter(prefix) {
    let sep = ": ";
    return fn(name) { return prefix + sep + name; };
}

let hello = makeGreeter("Hello");
Console.WriteLine(hello("World"));   // Hello: World
Console.WriteLine(hello("Flaris"));  // Hello: Flaris
```

Capture semantics - by value at bind time:

Integers, booleans, strings, and other scalar values are snapshotted when the inner function is created. Mutating the outer variable after the inner function is created does not affect the captured copy.

Assigning to a captured variable inside the closure changes only the closure's own copy. The new value persists across later calls of that closure, but the enclosing function never sees it:

```
fn makeCounter() {
    let n = 0;
    let f = fn() { n += 1; return n; };
    n = 100;          // f captured n = 0 above; this does not reach it
    return f;
}
let c = makeCounter();
Console.WriteLine(c());  // 1
Console.WriteLine(c());  // 2
```

Reference types (arrays, objects) share the same heap object - mutations are visible through the closure:

```
fn makeCounter() {
    let state = [0];
    return fn() {
        state[0] = state[0] + 1;
        return state[0];
    };
}

let c = makeCounter();
Console.WriteLine(c());  // 1
Console.WriteLine(c());  // 2
Console.WriteLine(c());  // 3
```

Loop snapshot capture:

When a closure is created inside a loop, the current value of the loop variable is captured at that iteration. Each closure gets its own independent snapshot:

```
fn makeFns() {
    let funcs = [];
    iter(i from 0 to 5) {
        Array.Append(funcs, fn() { return i; });
    }
    return funcs;
}

let fs = makeFns();
Console.WriteLine(fs[0]());  // 0
Console.WriteLine(fs[3]());  // 3
```

Globals are still accessible:

Module-level globals are always accessible from inner functions, as before:

```
global multiplier = 3;

fn makeTransform() {
    return fn(x) {
        return x * multiplier;
    };
}

let transform = makeTransform();
Console.WriteLine(transform(5));  // 15
multiplier = 2;
Console.WriteLine(transform(5));  // 10  (reads live global)
```

Variables and Fibers

Each fiber has its own call stack and local variables. Globals are shared:

```
global shared = 0;

fn worker(id) {
    let local = id * 10;  // local to this fiber's stack
    shared += 1;          // shared across all fibers
    yield local;
}
```

Common Mistakes

- `let x;` is illegal - always provide an initializer - forces developers to set a reasonable initial value
- `let` and `var` at the top level are illegal - use `global name = value;` at module scope
- `const` prevents rebinding the name and mutating the value it holds
- Declaring `let x` inside a function does not modify a global `x` - use `global x = value;` to declare a new global from within a function
- Reusing the same name in nested scopes shadows the outer variable
- `global x = value;` executed again for an existing global (a function that declares it, called twice) warns and redefines it - not a silent no-op; a `VM.Eval` unit may not redefine one at all (1004)

---

5. Expressions

An **expression** produces a value. Expressions become **statements** when terminated with `;`.

> Important: **Assignments do not produce a value at runtime.**

> In the grammar, `a = b` is parsed as an assignment expression node, but the compiler emits assignment bytecode that **does not leave a value on the stack**. Treat assignments as **statement-only**: you cannot write `let x = (y = 3)` or use `x = y` as a condition.

Literals

```
nil            // nil
true           // bool
42             // int
3.14           // float
"hello"        // string
'A'            // char  - single Unicode codepoint, single quotes
[1, 2, 3]      // array
{ a: 1, b: 2 } // object (keys are strings)
```

Char literals use single quotes. The `Char` module provides classification and conversion - see **Reference R8 - Char Module**.

Instance method syntax on strings, chars, and arrays

`String`, `Char`, and `Array` values support calling their built-in module methods directly on the value using dot notation. This is purely syntactic sugar - both forms are equivalent and can be mixed freely. When the variable is typed, the compiler resolves the call at compile time and emits identical bytecode to the module-style form.

```
// Char instance methods
let c = String.CharAt("hello", 0);
c.IsAlpha()       // true   - same as Char.IsAlpha(c)
c.ToUpper()       // 'H'    - same as Char.ToUpper(c)
c.Code()          // 104    - same as Char.Code(c)

// String instance methods
let s = "Hello, World!";
s.Length()        // 13     - same as String.Length(s)
s.ToLower()       // "hello, world!"
s.Contains("World")          // true
s.Substr(7, 5)               // "World"
s.Replace("World", "Flaris") // "Hello, Flaris!"
s.Split(",")                 // ["Hello", " World!"]
"  hi  ".Trim()              // "hi"

// Array instance methods (functions that take the array as first arg)
var a: array = [];
a = a.Append(1);
a = a.Append(2);             // same as Array.Append(a, 2)
a.First()                    // 1
a.Last()                     // 2
a.Contains(1)                // true
```

```
Math.Sum(a)                  // 3 - Sum is a Math function, not an array method
a = a.Reverse()
let evens = a.Where(fn(x) { return x % 2 == 0; });
let doubled = a.Select(fn(x) { return x * 2; });
```

Note: `Array.Create` and similar factory functions that do not take an array as first argument are not available as instance methods (`a.Create(5)` will not dispatch to `Array.Create`).

Operators

Arithmetic: `+` `-` `*` `/` `%` `^^`

- `/` yields float if any term is float, else int
- `^^` is exponentiation: `2 ^^ 3` - `8`. It binds tightest and nests to the right: `2 * 3 ^^ 2` is `18`, `2 ^^ 3 ^^ 2` is `512`, and `-2 ^^ 2` is `-4`
- An assignment is an expression: `a = b = 5` sets both, `let c = (a = 3)` stores and reads `3`. It is still an error as an `if`/`while` condition.
- `++`/`--` work on variables, fields and elements: `i++`, `o.n++`, `a[i]--`
- A `nil` operand on either side makes the result `nil`, for every arithmetic and bitwise operator: `nil + 5`, `"a" + nil` and `-nil` are all `nil`. A misspelled field (`o.cuont + 1`) gives `nil` too, not a number. Supply a default with `??` when you need one: `(o.count ?? 0) + 1`

Bitwise: `&` `|` `^` `~` `<<` `>>` `>>>` (and, or, xor, not, shl, arithmetic shr, logical shr)

- A cast to a sized int keeps the low bits: `(u8)x` gives what `x & 255` gives for an int, and `(i8)x` also sign-extends them, so `(i8)200` is `-56`. `(u16)`, `(i16)`, `(u32)` and `(i32)` do the same for 16 and 32 bits

Comparison: `==` `!=` `<` `<=` `>` `>=` `is`

- `x is C` tests for an instance of class `C`, subclasses included; with any other right side it compares identity (`x is nil`). It is not a type test: use `Type.IsInt(x)`, `Type.IsString(x)` and the other `Type.Is*` functions for that

Logical: `&&` `||` `!` - aliases: `and` `or`

- Short-circuiting: `x && y` only evaluates `y` if `x` is truthy
- Falsy: `nil`, `false`, `0`, `0.0`, `""`, `'\0'`, empty arrays/objects; everything else is truthy
- `x && y` and `x || y` always yield a bool, as in C#: `x && y` is `true` when both are truthy, `x || y` when either is. For a fallback value use `??`: `port ?? 8080`
- `and` / `or` are exact tokenizer aliases - identical precedence and behaviour to `&&` / `||`

Increment / Decrement: `++` `--` (prefix or postfix)

`++` and `--` work on variables (locals and globals), fields and array elements, in either position. They mutate in place and **produce no value** - they are statements and cannot be used inside expressions. The target must be numeric, or `nil` (it stays `nil`).

```
let i = 0;
let s = "";
```

```
i++;          // ok - i is now 1
++i;          // ok - same thing, i is now 2
obj.count++;  // ok
--arr[0];     // ok
let x = i++;  // ❌ compile error - ++ produces no value
s++;          // ❌ compile error - '++' requires a numeric type
```

Null coalescing: `x ?? y` - returns `y` only if `x` is `nil`

```
let name = user.name ?? "Anonymous";
```

Ternary: `condition ? exprIfTrue : exprIfFalse`

```
let label = age >= 18 ? "adult" : "minor";
```

Member Access

```
obj.key         // reads a property
obj.key = val;  // sets a property (statement)
```

Null-safe: accessing `.key` on `nil` returns `nil` - it does not throw.

```
let user = nil;
user.name;            // nil (safe)
user.items[0];        // nil (safe): the whole chain yields nil
user.Save();          // nil (safe): a call on nil yields nil; its arguments still run
let city = user.address.city ?? "none"; // "none" when any link is nil
```

Indexing

```
array[i]       // 0-based integer index
object["key"]  // string key (equivalent to object.key)
object[42]     // an int or char key names its text: object["42"]
string[i]      // byte at index i
block[i]       // element i of a block, as an int
```

A negative index counts from the end, as in Python: `a[-1]` is the last element and `a[-len(a)]` the first. This works for arrays, strings and blocks, both when reading and when assigning (`a[-1] = 0`). An index before the start or at or after the end raises `Exception.OutOfBounds`.

```
let a = [10, 20, 30];
a[-1];         // 30
a[-2] = 99;    // a is now [10, 99, 30]
"abc"[-1];     // 'c'
```

Null-safe: `nil[x]` returns `nil`.

A block element reads back **unsigned** when the block's elements are 1, 2 or 4 bytes wide, and as the plain 64-bit value when they are 8 bytes wide. A write keeps only the element's low bytes, so a byte written as `-1` reads as `255`:

```
let b = Buffer.Create(2, 1);
b[0] = 200;
b[1] = -1;
Console.WriteLine(b[0], " ", b[1]);         // 200 255
Console.WriteLine(Buffer.ReadI8At(b, 1));   // -1: the signed view
```

For signed data - PCM samples, signed fields in a binary format - read with `Buffer.ReadI8At`, `Buffer.ReadI16At` or `Buffer.ReadI32At` (byte offsets).

`guard`

`guard(expr)` asserts that `expr` is not `nil`. If it is, throws `Exception.GuardCheck`:

```
let addr = guard(user.address);  // throws if nil
```

Use `guard` when a nil value means something has gone wrong. For optional values, just use `??` or check directly.

`weak`

`weak(expr)` returns a handle on `expr` that does not keep it alive. Read it back with `Weak.Get`, which yields `nil` once the target is gone. It is the way to hold a back-reference without forming a reference cycle:

```
class Node {
    let child = nil;                // owned
    let parent = nil;               // observed
    fn Adopt(c) { this.child = c; c.parent = weak(this); }
    fn Up() { return Weak.Get(this.parent); }
}
```

Function Calls

```
fn add(a, b) { return a + b; }
let x = add(1, 2);   // 3
```

`await` and `yield`

```
let result = await asyncFn();   // wait for async function result
yield value;                    // suspend fiber, produce value
```

Note:

- `await expr;` used as a **statement** discards the resulting value.
- `let x = await expr;` used in an **expression** keeps the value.

Operator Precedence (high to low)

Level	Operators	Associativity
1	Literals, variables, `()`	-
2	`()` call, `[]` index, `.` member	Left
3	`^^` (power)	Right
4	`+` `-` `!` `~` `++` `--` `await` `new` `guard` (prefix)	Right
5	`*` `/` `%` `&` `|` `^` `<<` `>>` `>>>`	Left
6	`+` `-`	Left
7	`<` `<=` `>` `>=`	Left
8	`==` `!=` `is` `in`	Left
9	`&&`	Left
10	`||`	Left
11	`??`	Left
12	`?:` (ternary)	Right
13	`=` `+=` etc. (assignment)	Right

When in doubt, use parentheses.

---

6. Control Flow

Every statement ends with `;`. Newlines are not statement terminators.

If / Else

```
if (condition) {
    ...
} else if (other) {
    ...
} else {
    ...
}
```

Truthiness: `nil`, `false`, `0`, `0.0`, `""`, `'\0'` and empty arrays/objects are falsy; everything else is truthy.

While

```
let i = 0;
while (i < 10) {
    Console.WriteLine(i);
    i += 1;
}
```

Do / While

A `do { ... } while (cond);` loop tests its condition **after** the body, so the body always runs at least once. The trailing `;` is required.

```
let line = "";
do {
    line = Console.ReadLine();
    process(line);
} while (line != "quit");
```

For

```
for (let i = 0; i < 10; i += 1) {
    Console.WriteLine(i);
}
```

Any of the three clauses may be omitted. An empty condition means "always true", so `for (;;)` is a bare infinite loop (exit it with `break`, `return` or a `throw`):

```
for (;;) {
    let cmd = next();
    if (cmd == nil) { break; }
    run(cmd);
}

for (let i = 0; ; i += 1) {   // no condition — step and break inside the body
    if (i >= limit) { break; }
}
```

Foreach

Iterates arrays, objects, strings, and collections. A `nil` iterable runs the body zero times, like an empty one; any other value raises `Exception.InvalidArguments`.

```
foreach (item in [1, 2, 3]) {
    Console.WriteLine(item);
}

// With key
foreach (value, key in obj) {
    Console.WriteLine(key, "=", value);
}

// String characters
```

```
foreach (ch in "hello") {
    Console.WriteLine(ch);
}
```

Iterable	Key	Value
Array	integer index	element value
Object	property name (string)	property value
String	byte index	byte, as a `char`
Stack, Queue, Deque, Set	index	element (a snapshot taken when the loop starts)
HashMap, OrderedMap	key	value (OrderedMap: insertion order)

The loop variables need no annotation: over an `[int]`, `[float]`, `[char]` or `[bool]` array the value has the element type, over a `[[T]]` matrix it is a `[T]` row, over a string it is a `char`, and the index is an `int`. They cannot be assigned inside the body, but the value they name can be changed - `foreach (p in points) { p.x += 1; }` moves every point.

Iter

High-performance integer range loop - no allocation, optimized bytecode:

```
iter (i from 0 to 10) {
    sum += i;
}
```

- Only ascending (`from` < `to`)
- Bounds evaluated once before the loop

Use `iter` for tight numeric loops; use `foreach` for container iteration.

Break & Continue

```
for (let i = 0; i < 10; i += 1) {
    if (i == 5) break;     // exit loop
    if (i % 2 == 0) continue;  // skip to next iteration
    Console.WriteLine(i);
}
```

Switch

```
switch (value) {
    case 1:
        doOne();
        break;
    case 2:
    case 3:
        doTwoOrThree();
```

```
        break;
    default:
        doOther();
        break;
}
```

Cases fall through unless you `break`. `break` inside a switch does not affect outer loops.

Unlike C, `default` is always the **last** body regardless of where it appears in the source, so a case falling through reaches it after every other case body:

```
switch (x) {
    case 1: s = s + "1";
    default: s = s + "D";   // emitted last, whatever its position
    case 2: s = s + "2";
}
// x == 1 -> "12D"   (C would give "1D2")
```

Return

```
fn add(a, b) {
    return a + b;
}

fn log(msg) {
    Console.WriteLine(msg);
    return;   // returns nil
}
```

Exceptions

`Exception` is a built-in base **class**. Anything thrown is an `Exception` instance (or a subclass instance). Instances carry `Code` (int), `Error` (message string) and `StackTrace`, plus a `ToString()` method.

Throw:

```
throw new Exception(Exception.DivByZero, "Cannot divide by zero");

// Shorthand — desugars to `new Exception(code, msg)`:
throw(Exception.DivByZero, "Cannot divide by zero");
```

`throw` requires an `Exception` (or subclass) instance; throwing anything else is a type error.

Custom exception types - extend `Exception`:

```
class NetworkError : Exception {}
class TimeoutError : NetworkError {}

throw new NetworkError(503, "service unavailable");
```

A subclass that carries more data declares its fields and its own constructor, and passes the code and message on with `super(code, msg)`:

```
class ParseError : Exception {
    let line = 0;
    fn Constructor(msg: string, line: int) {
        super(Exception.InvalidArguments, msg);   // sets Code and Error, captures StackTrace
        this.line = line;
    }
}

try {
    throw new ParseError("unexpected '}'", 12);
} catch (e) {
    Console.WriteLine(e.ToString(), " at line ", e.line);   // ParseError (code 3): unexpected '}'
at line 12
}
```

Try / Catch / Finally:

```
try {
    let v = 1 / 0;
} catch (err) {
    Console.WriteLine("Error:", err.Error);
    Console.WriteLine("Code:", err.Code);
} finally {
    cleanup();   // always runs: normal exit, exception, return, break, continue
}
```

A `try` may have a `finally` and no `catch`: the block runs and the exception continues outward.

```
try {
    process();
} finally {
    cleanup();   // runs, then the exception (if any) keeps propagating
}
```

There is a single `catch` clause that binds the caught instance. To handle specific exception types, dispatch on the value with a type `switch` - `case <Class>:` is an `instanceof` test (subclass-aware), so order most-specific-first and use a bare `Exception` (or `default`) as the catch-all:

```
try {
    doRequest();
} catch (e) {
    switch (e) {
        case TimeoutError: retry();           break;
        case NetworkError: reconnect();       break;
        case Exception:    log(e.ToString()); break;   // any other exception
    }
}
```

Cases fall through like in C - end each case with `break` (or `return`) unless fallthrough is intended.

Re-raise from a catch with `throw e;`. The same `instanceof` test is available as the `is` operator: `if (e is NetworkError) { ... }`.

`finally` runs on every exit path - normal completion, a caught (or re-raised) exception, and `return` / `break` / `continue` that leave the `try` or `catch` (the leave runs every enclosing `finally` first, even across loop boundaries). If a `finally` itself throws, that exception supersedes any in flight.

Control may **not** leave a `finally` body via `return`, `break`, or `continue` (compile error) - a `finally` is cleanup that always runs to completion. A `throw` from a `finally` is allowed (it propagates to an upstream handler), and a `break` / `continue` targeting a loop opened *inside* the `finally` is fine (it does not leave the `finally`).

If no handler exists, the exception propagates up through fibers. An uncaught exception terminates the VM.

`case <ImportedClass>:` is an `instanceof` test exactly like a local class. Labels are tried top to bottom and the first match wins, so a base class listed before its subclass swallows it - here the `NetworkError` arm is unreachable, and the compiler warns that it can never match:

```
switch (e) {
    case Exception:    log(e);        break;   // matches every exception
    case NetworkError: reconnect();   break;   // never reached
}
```

Import

Flaris imports require **compiled bytecode** (`.flx`) and a **version requirement**:

```
import { sqrt } from library("./mathlib", "1.0");
import { sqrt, pow } from library("./mathlib", "1.0");
import { Foo as Bar } from library("./classes", "1");
```

Library names are case-sensitive

The first argument resolves to a file: `library("Signals")` loads `Signals.flx` from the `--libs` path, `library("./util/Math")` loads `./util/Math.flx`, and a URL loads that exact path. Resolution is an **exact,**

case-sensitive filename match - the name is never lowercased. Case-insensitive filesystems (macOS/Windows) forgive a mismatch, but on **Linux** `library("signals")` will not find `Signals.flx`. Keep the import string, the on-disk filename, and any published/pinned name identical; the convention is a capitalized first letter (`Signals`, `Http`, `BigInt`).

Imported **symbol names are case-sensitive too** and must match the library's `export` exactly - `import { Jwt }` matches `export { Jwt }`, not `JWT` or `jwt`.

Where libraries are found

A `library("Name")` import is resolved by trying, in order:

1. the name as a path - `library("./mathlib")` → `./mathlib.flx`, relative to the working directory;
2. each `;`-separated entry of `--libs=<paths>`;
3. the **per-user install directory** - `~/.flaris/libs` (POSIX) or `%USERPROFILE%\.flaris\libs` (Windows), overridable with the `FLARIS_LIBS` environment variable.

The first candidate whose embedded version satisfies the requirement wins, so an explicit `--libs` entry always takes precedence over a globally-installed copy. Step 3 means a package manager can install a library once into the home directory and have every script resolve it with **no `--libs` flag**.

The second argument is a version string matched against the version embedded in the `.flx` file at compile time. Set the version during compilation with `--min-version=` - the default is `1.0.0.0`.

Version syntax

String	Meaning
`"1"`	Major 1, any minor/patch/build
`"1.2"`	Major 1, minor 2, any patch/build
`"1.2.3"`	Major 1, minor 2, patch 3, any build
`"1.2.3.4"`	Exact match on all four components
`"=1.2"`	Exactly 1.2.x.x (explicit exact)
`">=1.2"`	Any version ≥ 1.2.0.0
`"<2.0"`	Any version < 2.0.0.0
`">=1.2 <2.0"`	Range: 1.2.0.0 ≤ v < 2.0.0.0

The **major version always requires an exact match** regardless of operator - a module compiled as `2.x` will never satisfy a `"1.x"` requirement. This enforces ABI stability.

Optional fingerprint verification

A third argument adds a tamper check: the **whole-file SHA-256** of the `.flx`, identical to `sha256sum file.flx`. Obtain it with `flarisvm --sha256 file.flx`:

```
import { Jwt } from library("./Jwt", "1.0", "a3f1...64hexchars...");
```

The hash is recomputed over the actual bytes loaded, so it detects any tampering. An optional `sha256:` prefix is accepted (the form flarispm records in `flaris.json`). If it does not match the loaded file the VM halts immediately, regardless of the global `--no-verify` flag. This is the recommended pattern for security-sensitive dependencies.

Compiling a library with a version

```
# Compile with version
flarisvm --compile mylib.fls mylib.flx --min-version=1.2.0

# Print fingerprint
flarisvm --sha256 mylib.flx
```

Only symbols explicitly `export`ed from a module are importable.

Imported types

Type-aware features - the `is` operator, a type `switch` (`case <Class>:`), and typed `catch` dispatch - need the compiler to know an imported symbol's *type*. Flaris resolves that two ways:

1. Automatically from the dependency's `.flx`. Each `export` records the exported symbol's type in the compiled module (functions, classes, constants, …). When you `import` from it, the compiler reads those types back, so an imported class is recognised as a class:

```
import { ApiError } from library("./errors", "1.0");   // ApiError is known to be a class

try { request(); }
catch (e) {
    switch (e) {
        case ApiError: handle(e);    // instanceof dispatch — resolved across modules
        case Exception: log(e);
    }
}
```

This requires the dependency's `.flx` to be **compiled before** the module that imports it.

2. Explicitly, with a `:type` annotation. You can also annotate the import, which always takes precedence (useful when the dependency isn't compiled yet, or to be explicit):

```
import { ApiError:class, MAX_RETRIES:int, connect:fn } from library("./net", "1.0");
```

Recognised type names: `nil`, `bool`, `int`, `float`, `char`, `string`, `array`, `object`, `class`, `instance`, `fn`, `module`, `fiber`, `stream`, `block`, `exception`, `any`, `number`.

If a symbol's type is unknown (no annotation and the dependency wasn't available at compile time), it is treated as `any` - code still runs correctly at runtime (type tests fall back to dynamic checks), it's just not statically resolved.

Static linking (self-contained bundles)

Pass `--bundle` when compiling to embed imported `.flx` files directly into the output. The result is a single `.flx` that runs anywhere without the dependency files present:

```
flarisvm --compile app.fls app_bundle.flx --bundle --libs='.'
flarisvm --exec app_bundle.flx          # mathlib.flx not needed on disk
```

Bare `--bundle` resolves the dependencies itself: every module the program `import`s, then every module those libraries import, and so on, each one looked up through the same search path an import would use (`--libs`, then `~/.flaris/libs`). The bundle therefore carries exactly the libraries the program reaches - no more, no less. A pinned import (`library("mathlib", "1.0", "sha256:...")`) is verified while building, so a wrong pin fails the build instead of the shipped artifact, and a URL import is downloaded and packed like any other dependency.

To choose the set by hand, name the files instead - the two forms can be combined, and a file named explicitly is never packed twice:

```
flarisvm --compile app.fls app_bundle.flx --bundle='mathlib.flx;util.flx' --libs='.'
```

Two things stay outside a bundle:

- **A module imported at runtime** through a direct `VM.Import(name, version)` call. Its name is a computed string, so no build-time walk can see it; the `.flx` must be shipped alongside and found through `--libs` or `~/.flaris/libs`. Only `import ... from library(...)` declarations are bundled.
- **Native libraries loaded with `Ffi`.** Those are OS shared objects opened at runtime; the build warns when a program uses `Ffi` so the dependency is not a surprise on the target machine.

A bundled dep is held to the same rules as an on-disk one, so an embedded binary (whose runtime always enforces `--require-signed`) needs its dependencies signed by a trusted key - see *Package signing* under §2.

Inspect the bundle:

```
flarisvm --disasm app_bundle.flx
```

Shows the TOC listing each bundled dep (name, version, offset, size) followed by a full disassembly of each chunk.

A bundled `.flx` still needs a VM to run it (`flarisvm` / `flaris`). To ship a single **native executable** with the runtime included, embed the same bundle with `--embed` (see *Self-contained binaries* under §2):

```
flarisvm --embed app.fls app --bundle
./app                                # standalone — no VM or deps needed
```

---

7. Functions

Functions are first-class values created with `fn`. The modifiers `static`, `async` and `inline` go either before `fn` or between `fn` and the name.

`<static> <async> <inline> fn name(<args:?type) ?:type` or `fn <static> <async> <inline> name(...)`

The same modifiers work on an anonymous function, where the name is simply absent: `fn async (x) { ... }` or `async fn (x) { ... }`.

Argument limit - functions accept up to 16. For larger configurations, pass an object:

```
// preferred for many params
fn draw(opts) {
    let x = opts.x;
    let color = opts.color;
    ...
}
```

Basic Declaration

```
fn greet(name) {
    return "Hello, " + name + "!";
}

greet("Alice");  // Hello, Alice!
```

Functions as Values

```
let f = greet;          // store in variable
f("Bob");               // call through variable

fn apply(func, val) {
    return func(val);
}
apply(greet, "Carol");  // pass as argument
```

Type Annotations (optional)

Type annotations on parameters and return values are checked at compile time where the types are known, and at run time where they are not:

```
fn add(a: int, b: int): int {
    return a + b;
}

add(5, 10);         // ✓ OK
add("x", "y");      // ❌ compile error 1000 – expected int, got string
add(Json.Parse(s), 1); // checked when called: raises InvalidArgs unless it is an int

fn count(o): int {
    return o.n;     // o is untyped: the returned value is checked, and a
}                   // non-int raises TypeMismatch (22)
```

Use `any` to explicitly accept all types for a parameter - this disables type checking for that parameter while still annotating the others:

```
fn log(label: str, value: any) {
    print(label, value);
}

log("result", 42);       // ✓ OK
log("result", [1,2,3]);  // ✓ OK — any type accepted
```

Array types nest one level for numbers, chars and bools: `[[float]]` is a matrix whose rows are `[float]` arrays, and code typed this way can run compiled.

```
fn trace(m: [[float]]): float {
    var s = 0.0;
    iter (i from 0 to len(m)) { s += m[i][i]; }
    return s;
}

let m: [[float]] = [[0.5, 9.0], [9.0, 1.25]];
trace(m);    // 1.75
```

A matrix only takes rows that are exactly `[T]` arrays; the reference lists the rules under Run-time checks.

Type annotations are **completely optional** - all existing code works without them. See **Reference R3** for the full type keyword list and union type syntax.

Parameters

A typed parameter is required: leaving it out is a compile error. Append `?` to a parameter name to make it optional. An untyped (or `any`) parameter is always optional. The VM passes `nil` for every omitted trailing argument. A `nil` passed to a required `int`, `float`, `char` or `bool` parameter arrives as that type's zero (`0`, `0.0`, `'\0'`, `false`): scalars are not nullable, every other type holds `nil` as a value, and `x: int|nil` receives the `nil` itself:

```
fn add(a: int, b: int) { return a + b; }

add(1, 2);   // OK
add(1);      // Error: Function 'add' expects 2 arguments, got 1.

fn show(a, b) { Console.WriteLine(a, " ", b); }

show(1);     // 1 nil — an untyped parameter may be left out


fn greet(name?: string) {
    Console.WriteLine("Hello " + (name ?? "stranger"));
```

```
}

greet("World");  // Hello World
greet();         // Hello stranger
```

Optional parameters must come after all required ones. They can be combined with union types:

```
fn log(msg: string, level?: string|nil) {
    // level is nil when omitted
}
```

For variable numbers of arguments, use an array:

```
fn sum(numbers) {
    let total = 0;
    foreach (n in numbers) { total += n; }
    return total;
}
sum([1, 2, 3, 4, 5]);
```

Return Values

Functions return `nil` if no `return` is given:

```
fn classify(x) {
    if (x < 0) return "negative";
    if (x > 0) return "positive";
    return "zero";
}
```

Async Functions

Async functions run as fibers and return immediately:

```
import { Http } from library("Http", "1.0");

fn async fetchData(url) {
    let response = await Http.Get(url);
    return response.body;
}

let fiber = fetchData("http://example.com");  // returns fiber
let data = await fiber;                        // wait for result
```

The call only creates the fiber; the body runs when the fiber is awaited (`await`, `Fiber.AwaitAll`). An async call used as a statement - `fetchData(url);` with no `await` and the fiber not kept - therefore never runs, and

the analyzer warns about it (code 2024).

Error handling in async:

```
fn async safeLoad(url): object|nil {
    try {
        let resp = await Http.Get(url);
        return resp.body;
    } catch (e) {
        Console.WriteLine("Error:", e.Error);
        return nil;
    }
}
```

Anonymous Functions

```
let square = fn(x) {
    return x * x;
};

Array.Select([1, 2, 3], fn(x) { return x * 2; });
```

Note: type annotations **are supported** on anonymous functions as well.

An anonymous function can be `async`, with the modifier on either side of `fn`. It behaves exactly like a named async function: calling it returns a fiber, and `await` works inside its body. This matters for callback-shaped code - a route handler, an event callback - where the function is written inline at the point it is passed:

```
app.Get("/report", fn async (req) {
    let r = await Https.GetAsync("https://api.example.com/report");
    return Response.Json(r.body);
});

let load = async fn (path) { return await File.ReadTextAsync(path); };
let text = await load("/etc/hosts");
```

`static` is not allowed on an anonymous function - it describes a class member, and there is nothing for it to attach to here.

Arrow Functions

Single-expression shorthand - the expression is automatically returned:

```
let double = fn(x) => x * 2;
let add = fn(a, b) => a + b;
```

```
fn square(x) => x * x;   // also works as named function
```

The body is one expression, so a `{` after `=>` opens an **object literal**, not a block: `fn(x) => { v: x }` returns an object. Use a full `fn(x) { ... }` body when you need statements. A named arrow function is a declaration and ends with `;`.

Recursion

```
fn factorial(n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Function Modifiers

Modifiers go before `fn` or right after it - both spellings mean the same:

```
async fn load() { ... }         // async fiber
static fn helper() { ... }      // class-level (no this)
fn inline calc(x) => x * 2;    // hint to inline

static async fn fetch(url) { ... }  // combine modifiers
```

`inline` is a hint: the compiler expands the body at each call site when it can do so without changing behavior - with or without `--no-opt` - and otherwise emits a normal call: when a name in the body would collide with a caller local, on deep inline recursion, when the call passes more arguments than there are parameters (too few is fine - the missing ones are `nil`), or when an argument is not provably of its parameter's declared type (the normal call is what checks it). Restrictions: the body may contain at most one `return`, and only as its last statement - a `return` inside a nested block (`if`, loop, `try`) is a compile error. Arguments are always evaluated exactly once, left to right. Note that `VM.PatchFunction` does not affect call sites that were inlined.

Common Patterns

Factory function:

```
fn createUser(name, age) {
    return {
        name: name,
        age: age,
        greet: fn() { Console.WriteLine("Hi, I'm " + name); }
    };
}
let user = createUser("Bob", 25);
user.greet();
```

Higher-order functions:

```
let nums = [1, 2, 3, 4, 5];
let doubled = Array.Select(nums, fn(x) => x * 2);
let evens   = Array.Where(nums, fn(x) => x % 2 == 0);
let total   = Array.Reduce(nums, fn(acc, x) => acc + x, 0);
```

Error handling with union return:

```
fn divide(a, b): number|nil {
    if (b == 0) return nil;
    return a / b;
}

let result = divide(10, 2);
if (result == nil) {
    Console.WriteLine("Cannot divide by zero");
}
```

---

8. Classes

Classes provide lightweight object-oriented structure. Flaris classes support instance methods, static methods, single inheritance, async methods, and reflection.

Basic Class

```
class Point {
    let x = 0;   // instance fields MUST be declared in the class body
    let y = 0;

    fn Constructor(x, y) {
        this.x = x;
        this.y = y;
    }

    fn move(dx, dy) {
        this.x += dx;
        this.y += dy;
    }

    fn ToString() {
        return "Point(" + str(this.x) + ", " + str(this.y) + ")";
    }

    fn static origin() {
```

```
        return new Point(0, 0);
    }
}

let p = new Point(10, 20);
p.move(5, -3);
Console.WriteLine(p.ToString());          // Point(15, 17)
Console.WriteLine(Point.origin().ToString());  // Point(0, 0)
```

Inside the class body you may declare:

- `fn Constructor(...)` - optional constructor
- `fn method(...)` - instance methods
- `fn static method(...)` - static methods
- `fn async method(...)` - async instance methods
- `let field = <const expr>` - **instance field default** (initializer must be a compile-time constant)
- `const NAME = <const expr>` - **static class constant** (stored on the class, not the instance)

Class body initializers

- Instance field defaults declared with `let` in the class body must be **compile-time constant expressions**.
- `const` declarations inside a class define **static members** (class-level constants).

Sealed instance fields

A class instance has a **fixed set of fields**: every instance field must be declared in the class body with `let` or `var` (own fields) or inherited from a base class. You **cannot** introduce a new field by assigning to it - not from the constructor, not from a method, and not from outside the instance:

```
class Point {
    let x = 0;
    let y = 0;

    fn Constructor(x, y) {
        this.x = x;      // OK - declared field
        this.y = y;      // OK - declared field
        this.z = 0;      // ERROR - 'z' is not a declared field
    }
}

let p = new Point(1, 2);
p.color = "red";         // ERROR - 'color' is not a declared field
```

This is enforced at three levels:

- **Compiler** and **analyzer** reject `this.X = ...` writes to an undeclared field at compile time (error `1014`). If the class extends a base that lives in an imported module (whose fields are not visible during

this compile), the static check is relaxed for that class and the runtime guard below applies.
- **VM** raises an exception at runtime if any write (`this.X = ...`, `instance.X = ...`, or a compound assignment) would create a field that is not declared on the instance's class or one of its bases.

To hold optional/late-bound data, declare the field up front with a `nil` default (`let handle = nil;`) and assign it later. If you genuinely need an open-ended bag of dynamic keys, use a plain **object literal** (`{}`) instead of a class instance - object literals remain fully dynamic.

Creating Instances

```
let p = new Point(10, 20);
```

1. Allocate a new object with class `Point`
2. Call `Constructor(10, 20)` if defined
3. Run the initializer block if the `new` has one (below)
4. Return the constructed instance

If `Constructor` is missing, the instance is created without initialization.

Initializer blocks

A `new` may end with a block that finishes the object off:

```
class Point {
    let x = 0;
    let y = 0;
    fn Constructor(a) { this.x = a; }
    fn scale(f) { return this.x * f; }
}

let n = 10;
let p = new Point(3) {
    y = scale(2);        // 6 — a bare name is this class's own member
    x = x + n;           // 13 — `n` is captured from here, by value
};
```

The block is a constructor body for the class you are constructing: `this` is the new instance even when the `new` sits inside another class, bare names are that class's own fields and methods (an inherited member still needs `this.`), and names from the surrounding scope are captured by value the way a closure captures them. `let` inside the block declares a block-local, and an exception thrown in the block propagates out of the `new`.

The Constructor

```
class User {
    let name = nil;
    let age  = 0;
```

```
    fn Constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
```

- Instance fields **must be declared** in the class body with `let` / `var`. The constructor assigns them; it cannot introduce new fields. See **Sealed instance fields** below.
- Any explicit `return` value is ignored - `new` always returns the instance
- If `Constructor` throws, construction fails

The Destructor

`Destructor` is called automatically when the last reference to an instance is released (ref-count drops to zero). Use it to release external resources such as file handles, network connections, or native memory blocks.

```
class FileHandle {
    let file = nil;
    fn Constructor(path) {
        this.file = Stream.Open(path, "w");
    }

    fn Write(text) {
        Stream.WriteString(this.file, text);
    }

    fn Destructor() {
        Stream.Close(this.file);
    }
}

fn Main() {
    let f = new FileHandle("out.txt");
    f.Write("hello");
    // f goes out of scope here — Destructor is called automatically
}
```

Rules and caveats:

- `Destructor` receives no arguments and its return value is ignored
- `this` refers to the instance being freed, exactly as in any other method
- If `Destructor` throws, the exception is swallowed and a warning is printed - it does **not** propagate
- `Destructor` is **not** inherited; each class must declare its own if needed
- Avoid storing `this` anywhere inside `Destructor` - resurrection (re-referencing the dying object) is not supported and results in a dangling pointer

Reference cycles

Flaris frees objects by reference counting, and nothing traces the heap afterwards. That makes destruction predictable, but it means **a cycle is never freed**: if two objects point at each other, neither count ever reaches zero.

The common shape is a child that keeps a reference back to its parent:

```
class Parser {
    let _sub = nil;                    // parent holds the child
    fn AddSub() {
        this._sub = new SubParser(this);
        return this._sub;
    }
    fn ProgName() { return "tool"; }
}

class SubParser {
    let _parent = nil;                 // ...and the child holds the parent
    fn Constructor(parent) {
        this._parent = parent;         // cycle — neither is ever freed
    }
    fn Name() { return this._parent.ProgName() + " sub"; }
}
```

Nothing here is reported at compile time and the program behaves correctly - it just never releases those objects.

Usually the back-reference exists only to read a value or two, so the fix is to copy what you need and drop the reference:

```
class SubParser {
    let _parentProg = "";
    fn Constructor(parent) {
        this._parentProg = parent.ProgName();   // copy the value, not the object
    }
    fn Name() { return this._parentProg + " sub"; }
}
```

When the child genuinely needs the live parent, pass it as an argument so the reference lasts only for the call, or clear the field in a teardown method before the structure is dropped. The same rule covers any loop - two objects referring to each other, or a closure that captures an object being stored back on it:

```
let self = this;
Fiber.Run(fn() { self.Loop(); });              // fine — Fiber.Run returns an int id
this._fiber = Fiber.New(fn() { self.Loop(); }); // cycle — the field holds a fiber
                                               // whose closure captures `self`
```

`Fiber.New` returns a `fiber` **object**, so keeping it in a field of the same object the closure captures closes the loop. `Fiber.Run` hands back an integer id instead, which holds nothing and is always safe to store.

Check with `--mem`, which prints a leak dump at shutdown and exits non-zero when anything is left over:

```
flarisvm --mem myprogram.fls
```

A non-zero leak count that stays the same across runs is almost always a cycle. See Reference R7 "Memory Management" for the underlying model.

Inheritance

Single inheritance with `:`:

```
class Animal {
    let name = nil;
    fn Constructor(name) {
        this.name = name;
    }
    fn speak() {
        return this.name + " makes a sound";
    }
}

class Dog : Animal {
    fn speak() {
        return this.name + " barks";   // 'name' inherited from Animal
    }
}

let d = new Dog("Rex");
d.speak();   // "Rex barks"
```

Method lookup walks: instance - derived class - base class - ...

Calling the base constructor:

```
class ColorPoint : Point {
    let color = nil;
    fn Constructor(x, y, color) {
        super(x, y);          // calls Point.Constructor(x, y)
        this.color = color;
    }
}
```

Calling an overridden base method:

`super.method(args)` calls the base-class implementation from inside an override. The lookup is static: it resolves on the base of the class that *defines* the current method (not the receiver's dynamic class), so a multi-level chain of overrides each reaches its own base without looping:

```
class Animal {
    fn describe() { return "animal"; }
}

class Dog : Animal {
    fn describe() { return super.describe() + " that barks"; }
}

let d = new Dog();
d.describe();   // "animal that barks"
```

Calling a method that does not exist on the base chain raises a runtime error (a super call never falls back to the receiver's own class - that would re-enter the override). `super.method` is only available inside instance methods of a class with a base class.

Static Methods

Static methods belong to the class, not instances:

```
class MathUtils {
    fn static abs(n) {
        if (n < 0) return -n;
        return n;
    }
}

MathUtils.abs(-42);   // 42
```

Static methods are inherited and can be overridden.

Async Methods

```
import { Http } from library("Http", "1.0");

class Loader {
    let data = nil;
    fn async load(url) {
        let data = await Http.Get(url);
        this.data = data;
        return true;
    }
}
```

```
let l = new Loader();
let ok = await l.load("http://example.com");
```

`this` is preserved across `await` because it lives in the fiber's call frame.

Bound Methods

When you read a method from an instance, you get a bound method that remembers its receiver:

```
let m = p.move;   // bound to p
m(5, 10);         // equivalent to p.move(5, 10)
```

This makes callbacks natural - you can pass `obj.method` directly.

Classes as Values

Classes are objects and can be stored and passed around:

```
let C = MyClass;
let obj = new C();
C.Version = "1.0";   // add static fields dynamically
```

Because the class in a `new` expression is just a value, it may be reached through a dotted path - a class held in an object field or a namespace-like container:

```
let registry = { Widget: MyClass };
let w = new registry.Widget(1, 2);   // new on a member path
```

Reflection

```
Class.IsClass(val)          // true if val is a class
Class.IsInstance(val)       // true if val is an instance
Class.Name(MyClass)         // "MyClass"
Class.InstanceOf(obj, MyClass)   // inheritance check
Class.SubclassOf(Child, Parent)  // class hierarchy check
Class.GetBase(MyClass)      // parent class
Class.InstanceMethods(MyClass)   // ["Constructor", "move", ...]
Class.StaticMethods(MyClass)     // ["origin", "create", ...]
Class.Invoke(target, "method", args...)  // dynamic method call
Class.Fields(instance)      // ["x", "y", "color"]
```

Object helpers:

```
Object.Clone(value)              // deep clone
Object.HasKey(obj, key)          // own-property check
Object.Count(obj)                // number of own fields
Object.Entries(obj)              // [[key, value], ...]
```

```
Object.TryGet(obj, key, default) // safe lookup
Object.Clear(obj)                // remove all fields
```

Common Mistakes

- Forgetting `new` - `MyClass()` is not the same as `new MyClass()`
- Expecting multiple inheritance - only single inheritance is supported
- `return` in `Constructor` is ignored - `new` always returns the instance
- `Class.InstanceOf(instance, class)` - argument order matters
- `Class.SubclassOf(child_class, parent_class)` - argument order matters
- Storing `this` inside `Destructor` - resurrection is not supported; the object is freed immediately after `Destructor` returns
- Expecting `Destructor` to be inherited - each class must declare its own

---

9. Fibers & Async

Fibers are lightweight cooperatively-scheduled execution contexts. They allow concurrency, coroutines, and async/await without OS threads.

Key properties:

- Cooperative with a safety net - a fiber runs until it yields, awaits, returns or sleeps, or until its scheduling quantum expires and the scheduler re-queues it behind the other ready fibers
- No data races by default
- Cooperative scheduling

Concurrency model

Flaris supports two complementary models that run on the same scheduler:

Fibers - manual, controllable coroutines:

- `yield` produces values
- `Fiber.Resume` advances a fiber one step at a time

Async/await - automatic, high-level tasks:

- `fn async` creates a fiber-based task
- `await` runs it to completion and returns the final result
- `yield` inside async functions is a scheduler hint, not visible to the caller

---

Creating Fibers

Explicitly:

```
let f = Fiber.New(myFunction);
```

Implicitly via async functions:

```
fn async load() { return 42; }
let f = load();   // creates a fiber, does not run yet
```

A newly created fiber does not run until resumed.

Waiting for several fibers

Because a fiber starts only when it is first resumed or awaited, awaiting fibers one by one runs them one after another:

```
fn async fetch(url) { /* ... */ }

let a = await fetch(urlA);   // fetch(urlB) has not started yet
let b = await fetch(urlB);
```

`Fiber.AwaitAll` starts them all first, so their waits overlap, and returns every result in the order given:

```
let results = Fiber.AwaitAll([fetch(urlA), fetch(urlB)]);
```

If any of them fails, `AwaitAll` raises the first failure in array order once all of them have finished.

Resuming

`Fiber.Resume(f)` - runs the fiber until it yields or returns, then resumes the caller:

```
fn counter() {
    yield 1;
    yield 2;
    return 3;
}

let f = Fiber.New(counter);
Fiber.Resume(f);   // 1
Fiber.Resume(f);   // 2
Fiber.Resume(f);   // 3
```

`Fiber.Resume(f, true)` - **detached** mode, runs fiber in background:

```
Fiber.Resume(f, true);  // caller continues immediately, fiber runs independently
```

Fiber States

`Fiber.Status(f)` returns one of these, also available as `Fiber.Status.New` and so on:

State	Meaning
New (0)	Created, not started yet
Running (1)	Currently executing

State	Meaning
Suspended (2)	Not running: sleeping (`Fiber.Sleep`), parked (`Fiber.Park`), or queued to run again
Finished (3)	Completed
WaitIO (4)	Waiting for I/O
Yield (5)	Yielded a value
WaitFiber (6)	Waiting for another fiber

Yield

`yield value` suspends the fiber and returns the value to whoever called `Fiber.Resume` :

```
fn producer() {
    yield "first";
    yield "second";
    return "done";
}
```

Inside async functions, `yield` acts as a scheduler hint - it does not affect the `await` return value.

Await

`await f` waits for fiber `f` to complete and returns its final `return` value (not yield values):

```
fn async compute(x) {
    yield x;         // ignored by await
    return x * 2;
}

let v = await compute(5);  // 10
```

Awaiting an async function:

```
fn async load() { return 123; }
let v = await load();   // 123
```

Non-blocking built-ins

Several standard-library functions have an `Async` variant that offloads blocking work to a background thread so other fibers keep running:

Sync	Async	Module
`File.ReadText(path)`	`File.ReadTextAsync(path)`	File
`File.ReadAllBytes(path)`	`File.ReadAllBytesAsync(path)`	File
`File.WriteText(path, text)`	`File.WriteTextAsync(path, text)`	File
`Net.ResolveDNS(host)`	`Net.ResolveDNSAsync(host)`	Net
`Os.Execute(cmd)`	`Os.ExecuteAsync(cmd)`	Os
`Os.WaitTimeout(pid, ms)`	`Os.WaitAsync(pid, ms?)`	Os

Usage - call with `await` inside an async function:

```
fn async loadFiles(a, b) {
    let text  = await File.ReadTextAsync(a);
    let bytes = await File.ReadAllBytesAsync(b);
    return text;
}
```

The calling fiber suspends at each `await` and is resumed automatically when the operation completes. Other fibers continue to run in the meantime.

An `*Async` call has to be awaited right where it is made - `await File.ReadTextAsync(p)` or `Fiber.Await(File.ReadTextAsync(p))` - to get its result. Called on its own, as a statement, it starts the operation and carries on at once: the work still happens (`Os.ExecuteAsync("cmd")` runs the command in the background) and its result is discarded. Keeping the call and awaiting it later does not work, because the call itself returns `nil`. To start work now and collect it later, put it in an async function - calling one returns its fiber:

```
fn async readLater(path) { return await File.ReadTextAsync(path); }

let job = readLater(path);   // starts now
// ... other work ...
let text = await job;
```

Sleep

```
Fiber.Sleep(500);   // sleep 500ms, non-blocking for other fibers
```

Other fibers continue to run while one is sleeping.

Waiting from a plain function

`await` is only allowed inside an `async` function. `Fiber.Await` does the same thing as a call, so code that is not async - a request handler chain, a callback - can still wait for a fiber without blocking the VM:

```
fn async fetch(url) { return await Https.GetAsync(url); }

fn handler(req) {                              // not async
    let r = Fiber.Await(fetch(req.query.url));   // parks this fiber only
    return r.body;
}
```

It also accepts the `nil` an `*Async` built-in returns, so `Fiber.Await(File.ReadTextAsync(path))` reads a file from a plain function. An exception that escapes the awaited fiber is raised at the `Fiber.Await` call.

Error Handling

An exception that is not caught inside a fiber is delivered to whoever is blocked on it - `await f`, or an attached `Fiber.Resume(f)`:

```
fn async fail() {
    throw(42, "something went wrong");
}

try {
    await fail();
} catch (e) {
    Console.WriteLine(e.Error);  // "something went wrong"
    Console.WriteLine(e.Code);   // 42
}
```

The instance arrives intact - subclass, `Code`, `Error`, and the `StackTrace` captured at the throw site inside the fiber - so a type `switch` dispatches on it exactly as it would on a local `throw`:

```
class NetworkError : Exception {}

fn async request() { throw new NetworkError(503, "service unavailable"); }

try {
    await request();
} catch (e) {
    switch (e) {
        case NetworkError: reconnect();       break;
        case Exception:    log(e.ToString()); break;
    }
}
```

Delivery chains: if the awaiting fiber has no handler either, the exception keeps propagating outward to whoever is awaiting *it*.

A fiber nobody is waiting on cannot deliver to anyone. An uncaught exception in a detached fiber (`Fiber.Resume(f, true)`, `Fiber.Run(...)`), or one that reaches the top of the main fiber, terminates the VM with a stack trace and an exit code taken from the exception. If a detached fiber can fail, catch it inside the fiber:

```
fn async worker() {
    try   { doWork(); }
    catch (e) { Log.Error(e.Error); }   // nobody upstream to receive it
}
```

Example: Interleaving

```
fn async worker(name, n) {
    let i = 0;
    while (i < n) {
        Console.WriteLine(name, i);
```

```
        yield;
        i += 1;
    }
}

let f1 = worker("A", 2);
let f2 = worker("B", 2);

// Scheduler interleaves them:
// A 0 | B 0 | A 1 | B 1
let r1 = await f1;
let r2 = await f2;
```

Example: Async Chain

```
fn async inner(x) {
    yield x;
    return x * 2;
}

fn async outer(x) {
    let z = await inner(x);
    return z + 1;
}

Console.WriteLine(await outer(5));  // 11
```

Common Mistakes

- `await` returns the final `return` value - not `yield` values
- Fibers don't run until resumed - create + schedule explicitly
- Detached fibers have no one to deliver an exception to - an uncaught one ends the VM. Catch inside the fiber (an awaited fiber can hand it to its awaiter)
- A tight loop only hands over when its quantum expires - `yield` explicitly where latency matters
- `yield` outside a fiber context is an error

---

10. Working with JSON

The `Json` module is built in - no import. It follows the JSON standard strictly and never guesses: invalid input gives `nil`, and `Json.LastError()` tells you why and where.

Parse and serialize

```
let order = Json.Parse("{\"id\": 7, \"items\": [{\"sku\": \"A1\", \"price\": 9.5}]}");
Console.WriteLine(order.items[0].sku);          // A1
```

```
let text = Json.Stringify(order);              // compact
let pretty = Json.Stringify(order, true);      // 2-space indent
let custom = Json.Stringify(order, { indent: 4, sortKeys: true });
```

Numbers keep their kind: `7` parses as an `int`, `9.5` as a `float`, and a float is written so it parses back as the same float (`1.0` stays `1.0`, `0.1` stays `0.1`). A value that has no JSON form - NaN, a function - becomes `null`.

When parsing fails

`Parse` returns `nil` for invalid input - and also for the valid document `null`. When the difference matters, check the error:

```
let v = Json.Parse(body);
if (v == nil) {
    let e = Json.LastError();
    if (e != nil) {
        Console.WriteLine("bad JSON at line ", e.line, ", column ", e.column, ": ", e.message);
        return nil;
    }
}
```

A document that is valid but too large (over 4 million values, or a string over 128 MB) raises instead of returning `nil`, so it cannot pass as "no data".

Mapping JSON onto a class

`Deserialize` fills the fields of a class by name - handy for request bodies and config files. Each value must fit the field's type, which is taken from the field's default:

```
class Item {
    let sku = "";
    let price = 0.0;
    let tags: [string] = nil;
}

let items = Json.Deserialize("[{\"sku\": \"A1\", \"price\": 9}]", Item);
Console.WriteLine(type(items[0].price) == Type.Float); // true — an int fits a float field

let bad = Json.Deserialize("{\"price\": \"cheap\"}", Item);
Console.WriteLine(bad == nil, " ", Json.LastError().message);
// true field 'price' expects a number, got a string
```

Unknown keys are ignored and missing ones keep their defaults; the constructor is not run. `Json.Stringify(instance)` writes the fields back out.

Reaching into a document

Paths use JSONPath syntax (or a JSON Pointer when they start with `/`):

```
let doc = Json.Parse(text);
Json.Find("items[0].sku", doc);                // "A1"
Json.Find("items[-1].price", doc);             // last item
Json.Find("items[*].sku", doc);                // every sku, as an array
Json.Find("$..price", doc);                    // every price, at any depth
Json.Find("/items/0/sku", doc);                // JSON Pointer
Json.Exists("items[*].discount", doc);         // false unless one has it

Json.Set(doc, "meta.source", "import");        // creates `meta`
Json.Remove(doc, "items[0]");
```

`Json.FastSelect(path, text)` answers a path straight from the text without building the whole tree - the fast choice for pulling one field out of a large response you otherwise do not need.

Changing documents

```
// RFC 7386 merge-patch: null deletes, objects merge, the rest replaces
Json.Merge(settings, { theme: "dark", beta: nil });

// RFC 6902 JSON Patch: returns a patched copy, or nil if any step fails
let next = Json.Patch(doc, [
    { op: "replace", path: "/items/0/price", value: 8.5 },
    { op: "add", path: "/items/-", value: { sku: "B2", price: 3.0 } }
]);
```

`Set`, `Remove` and `Merge` change the value in place and refuse (raising `ConstAssign`) to touch a `const` or `Object.Freeze`d value; `Patch` works on a copy, so it can take a frozen one.

Canonical form, formatting, NDJSON

- `Json.Canonicalize(v)` - RFC 8785 canonical JSON: the same bytes for equal values however they were built, for hashing and signatures.
- `Json.Format(text)` / `Json.Minify(text)` - re-lay out JSON text without changing a single number or escape; `nil` if the text is not valid JSON.
- `Json.ParseLines(text)` / `Json.StringifyLines(array)` - newline-delimited JSON (one value per line), as used by log pipelines and streaming APIs.

---

11. Analyzer & Diagnostics

After parsing, Flaris runs a semantic analyzer. It resolves names, builds scopes, validates language rules, and emits helpful errors and warnings before bytecode is generated.

Diagnostics are reported like:

```
❌ Error: file:line:column: 1003: 'x' is not defined.
⚠️ Warning: file:line:column: 2000: Local 'x' is never read (prefix with '_' to suppress).
```

`CODE` is a stable number - errors are 1000+, warnings 2000+. The full table is in the reference under *R12 - Static Analyzer*. Diagnostics go to stderr, sorted by source position.

- **Errors** stop compilation (after a limit).
- **Warnings** allow compilation to continue.

Common Errors and Fixes

Undefined symbol / function

Symptoms

- `'x' is not defined.`
- `Call to undefined function 'foo'.`

Fix

- Declare it (`let x = ...;`) in an accessible scope.
- Check spelling and casing.
- If it is a method, call it on the receiver: `obj.method()` .

Redeclaration

Symptom

- `Redeclaration of 'name'.`

Fix

- Rename the inner declaration, or remove `let` and reassign instead.

Invalid `this`

Symptom

- `` `this` is not available here. ``

Fix

- Use `this` only inside **instance** methods (not static functions).

Invalid `super`

Symptom

- `` `super` is not available here. ``

Fix

- Use `super(...)` only inside an instance constructor, and `super.method(...)` only inside instance methods, of a class that has a base class.

`await` outside async

Symptom

- `` `await` used outside async function. ``

Fix

- Mark the containing function as `fn async ...`, or remove `await`.

Assigning to `const`

Symptom

- `Cannot modify const 'X' or its properties.`

Fix

- `const` is deep: neither the binding nor its properties or elements can be changed, so `cfg.port = 1;` on a `const cfg` is this error too.
- Use `let` for a value you rebind or mutate.

Missing return on some paths

Symptom

- `Function 'name' has a declared return type but can finish without returning a value.`

Fix

- Add `return` in every branch, or allow `nil` return if that is intended.

Value may be nil where a non-nil type is required

Symptom

- `1000: Type mismatch for 'b' (expected compatible with declared type).` on a read of a `T|nil` value.

Fix

- Prove it is not nil first. The analyzer narrows the value for the region the test covers - the guarded branch, or the rest of the block after an early exit:

```
fn use(a: int|nil) {
    let bad: int = a;              // 1000 — a may be nil here

    if (a != nil) {
        let ok: int = a;           // narrowed inside the branch
```

```
    }

    if (a == nil) { return; }
    let alsoOk: int = a;           // narrowed for the rest of the block
}
```

`a == nil`, `a != nil` (either operand order), `is_nil(a)`, `!`, and `&&` / `||` combinations are all recognised. Narrowing applies to plain variables only, not to `a.b` or `a[i]`. If none of these fit, cast: `let ok: int = (int)a;`.

Assignment used as a condition

Symptom

- `Assignment used as the 'if' condition; use '==' to compare.` (also `'while'` and `'for'`)

Fix

An assignment that is the whole condition is almost always a typo for `==`. To assign and test in one step, make the assignment an operand of the test:

```
// bad
if (x = get()) { ... }

// good
while ((line = Next()) != nil) { ... }
```

Common Warnings and Fixes

Unused variable / parameter

Symptoms

- `Local 'x' is never read (prefix with '_' to suppress).`
- `Global 'x' is never read.`
- `Parameter 'p' is never used (prefix with '_' to suppress).`

Fix

- Use it, remove it, or prefix the name with `_` to document that it is intentionally unused. The `_` prefix works for locals, parameters and globals alike. Loop and `catch` variables are exempt automatically.
- To silence one specific warning instead, add `// flaris-ignore[unused-symbol]` on the line (or on the line above it), or pass `-Wno-unused-symbol`. Errors cannot be silenced - see *R12 - Static Analyzer* in the reference.

Unreachable code

Symptom

- `Unreachable code.`

Fix

- Remove code after `return`, `throw`, or other guaranteed exits, or restructure with a condition.

Shadowing

Symptom

- `'x' shadows declaration at line N.`

Fix

- Rename the inner variable to avoid confusion.

Class member or constructor does not exist

Symptoms

- `Class 'Counter' has no method 'Nope'.`
- `Class 'Counter' has no member 'missing'.`
- `Constructor of 'Counter' expects 0–1 arguments, got 3.`

Fix

- Instances are sealed: declare the field with `let` in the class body, or fix the name. These are only reported when the receiver's class is provable - `this`, a local from `new C()`, or a parameter annotated `v: C`.

Circular inheritance

Symptom

- `Class 'A' is part of a circular inheritance chain.`

Fix

- Break the cycle. Base classes may be declared in any order, so this is a real cycle, not a forward reference. A cycle cannot be left to the runtime: method lookup would walk the base chain forever.

Operator does not accept an operand type

Symptom

- `Operator '-' does not accept string as its left operand.`
- `Operator '<<' does not accept float as its left operand.`

Fix

- Convert explicitly - `(int)x`, `Convert.ToInt(s)`. Note that `<<`, `>>`, `>>>`, `&`, `|` and `^` run on integers only and never widen to float. `+` is never reported: it falls back to string concatenation for any operand pair without a `nil`, and a `nil` operand makes any arithmetic result `nil`.

12. Debugging

Flaris debugging is built directly into the language and VM. No external debugger required - and no separate adapter process either: the VM speaks the Debug Adapter Protocol itself, so an editor talks straight to it.

In VS Code

Install the extension from `vscode-flaris/` and press `F5` on any `.fls` file. Breakpoints go in the gutter, `F10` / `F11` step, and the Variables pane expands objects, class instances and arrays.

Two things are worth knowing because they have no equivalent in most debuggers:

- **Each fiber is a thread.** The editor's thread picker is a fiber picker - you can read the stack of any parked fiber while the program is stopped.
- **Right-click a frame in the Call Stack → Open Disassembly View** to see the bytecode around it.

Breakpoints can be added and removed while the program runs, and the pause button interrupts it at the next source line. A `launch.json` is only needed to change defaults:

```
{
  "type": "flaris",
  "request": "launch",
  "name": "Debug current Flaris file",
  "program": "${file}",
  "cwd": "${workspaceFolder}",
  "vmArgs": ["--libs=./libs"]
}
```

Any other DAP-capable editor can attach with `flarisvm --dap=<port> app.fls`, which listens on loopback and waits for a client before running anything.

Everything below works the same from a terminal, which is what you want over SSH or in a container.

Debug Symbols

Debug info maps VM instructions back to source lines. It is **off** by default; `-g` (alias `--no-strip`) turns it on, and `--debug` / `--dap` turn it on for you.

```
flarisvm script.fls           # no debug info (default)
flarisvm -g script.fls        # line numbers in stack traces
```

Function and file names are always recorded, so a stack trace names every frame either way. Without `-g` every frame reports line 0; with it, the real line (lines past 65,535 all report 65,535).

Overhead: a compiled `.flx` grows by typically 7-17%, and each source line in a hot loop costs one extra dispatch per iteration when the code is interpreted.

Breakpoints

Run with `--debug` and the program stops at the start of `Main`, where you can set breakpoints by function or line and step through from there - no source edits needed:

```
$ flarisvm --debug app.fls
[Entry] Main (app.fls:40) [fiber 1]
(fdb) b processData        # or: b 42, or b app.fls:42, or b Order.Describe
breakpoint 1 at app.fls:12 (processData)
(fdb) c

[Breakpoint 1] processData (app.fls:12) [fiber 1]
(fdb) n                    # step over; s steps in, finish runs to the return
[Step] processData (app.fls:13) [fiber 1]
(fdb)                      # an empty line repeats the last step
```

`--debug` disables the JIT: a JIT-compiled function runs natively and never reports its lines, so breakpoints inside one could not fire.

If an exception is never caught, `--debug` opens the prompt at the throw site with every frame still standing, so you can read the values that caused it:

```
[Unhandled exception: code 42] level3 (app.fls:3) [fiber 1]
(fdb) locals
  [0] x                = (int) 11
  [1] doomed           = (int) 22
```

Without `--debug` the same failure is reported with its own code and message and the process exits with that code. `fibers` lists every live fiber and what it is waiting on, which is usually what you need when a cooperative program stops making progress rather than crashing.

You can also stop from the source itself. `breakpoint <code>;` pauses execution at that point:

```
fn processData(items) {
    breakpoint 100;    // entry

    for (let i = 0; i < len(items); i += 1) {
        if (items[i] < 0) {
            breakpoint 200;  // triggered for negative values
        }
    }

    breakpoint 300;    // exit
}
```

Reaching one opens a prompt where you can walk the call stack and inspect values, then continue:

```
[Breakpoint:100] processData (app.fls:42) [fiber 1]
(fdb) locals
  [0] items            = (array) [4, -7, 12]
  [1] i                = (int) 1
(fdb) p items[1]
items[1] = (int) -7
(fdb) bt
-> #0  processData (app.fls:42)
   #1  Main (app.fls:9)
(fdb) c
```

`bt` walks the stack, `up` / `down` pick a frame, `locals` and `p <path>` inspect it, `list` shows the surrounding source, `dis` shows the bytecode around the instruction the frame is stopped at (and `dis ir <fn>` the JIT IR of a compiled function), and `help` lists everything. Inspection never changes the program: the prompt allocates nothing and raises nothing.

The prompt opens when stdin is a terminal, so a `breakpoint` left in committed code prints its location and continues under a pipe or in CI instead of hanging. Pass `--debug` to open it anyway. With `--verbose` the surrounding bytecode is shown as well. Use meaningful codes to track investigation stages.

Conditional breakpoints: no special syntax is needed - the condition is ordinary code, so any expression works:

```
if (data[i] == nil) {
    breakpoint 200;  // only breaks when data is nil
}
```

The Debug Module

```
Debug.Value(obj)          // deep inspection: type, refs, address, value
Debug.Assert(actual, expected)  // halt with diff if not equal
Debug.Pool()              // memory allocator statistics
Debug.StackPtr()          // current stack depth (int)
Debug.Refs(obj)           // reference count of obj
Debug.GuardAddress(addr)  // software memory watchpoint
Debug.Here(msg?)          // print current file:line:function
```

`Debug.Value` example:

```
let user = { name: "Alice", age: 30 };
Debug.Value(user);
// DBG-VAL: [0x7f8a4c002a40] type object. Refs 1. Value: { name: "Alice", age: 30 }
```

`Debug.Assert` example:

```
Debug.Assert(result, 42);
// On failure: ✗ Assertion failed on line 56: Left: 43  Right: 42
```

`Debug.GuardAddress` - software memory watchpoint:

```
Debug.GuardAddress(data);   // any read/write to this address triggers a pause
process(data);
```

`Debug.Here` - quick location tracing:

```
fn complexLogic() {
    Debug.Here();               // prints file:line:function
    Debug.Here("inside if");    // with optional message
}
```

Verbose Modes

```
flarisvm script.fls           # normal (errors + output only)
flarisvm script.fls --verbose        # verbose (timing, info, function entry)
flarisvm script.fls --trace       # very verbose (every opcode)
flarisvm script.fls --verbose --time     # verbose + timing report
flarisvm script.fls --verbose --stats     # verbose + allocation statistics
```

Typical Workflows

Development debugging:

```
fn investigate() {
    let data = loadData();
    Debug.Value(data);              // inspect loaded state
    breakpoint 100;                 // pause here
    Debug.GuardAddress(data);       // watch for corruption
    process(data);
    Debug.Assert(data.valid, true); // verify post-condition
}
```

Memory leak detection:

```
flarisvm script.fls --stats
```

```
let before = VM.CurrentAllocations();
repeat1000Times();
VM.CompactMemory();
let after = VM.CurrentAllocations();
```

```
if (after > before + 10) {
    Console.WriteLine("Leak! Delta:", after - before);
}
```

Performance profiling:

```
flarisvm script.fls --time           # [Time] Execution took 34.943 ms. Idle time 0ms
flarisvm script.fls --prof           # where the time went, by function
flarisvm script.fls -g --prof=500    # by source line, 500 samples a second
```

`--prof` samples the running script from a CPU timer and prints a table at exit:

```
[Profile] 1000 Hz: 812 samples on script code (812.0 ms), 2 in the VM, 40.0 ms waiting, 855.0 ms
wall
[Profile] objects: 12034 allocated, 61 live at peak, 12 live at start
     %      self     total   objects  function                          location
interpreted because
  91.3%       741       741     12000  hot                               script.fls:4           a
parameter is untyped, or not a JIT type (...)
   8.7%        71       812        20  Main                              script.fls:11
reads a global variable
```

`self` is the samples taken while the function was running, including the time it spent inside a builtin or a library call, so the column answers "which of my functions pays" rather than "which native routine ran". `total` adds the samples taken while the function was anywhere on the call stack, so a caller is charged for what it calls; a recursive function counts once per sample. `objects` is the number of objects allocated while the function was on top, charged the same way as the samples. The milliseconds come from the clock, so they stay right when the OS delivers fewer ticks than asked for (the header then says how many it delivered). Samples "in the VM" are ticks with no fiber running: loading and the scheduler. "waiting" is wall time with no CPU use: I/O waits and sleeps. When more than one fiber took samples, a second table lists the fibers by their entry function.

A function the JIT has compiled to native code has no sampling points of its own, so its time is charged to the interpreted function that called it; run with `--jit-disable` when you want exact attribution.

Every row is therefore a function that ran in the interpreter, and the last column says why - the same reason `--check` gives, without a second run:

Column says	Meaning
a `--check` reason	The function is not JIT-eligible; fix what it names to make it native
`compiled, but these calls failed the native entry check`	Native code exists, but the arguments these calls passed did not match it - most often a `[int]` / `[float]` parameter given an array that is not internally typed - so they ran interpreted
`eligible, but the backend did not compile it`	The analyzer accepted the function and the native compiler declined it, for example because its body is too large
`top-level code is never compiled`	Statements outside any function

Column says	Meaning
`not JIT-eligible; --check on the source says why`	The code was loaded from a `.flx`, which does not keep the reason

With `--jit-disable` the column is left out, since everything is interpreted by request. Lines need debug info (`-g`); without it the table shows files only. The profiler is part of `flarisvm`, not of the runtime-only `flaris`. macOS and Linux sample from the process CPU timer; Windows ticks a sampler thread at up to 1000 Hz and counts a tick only when the script thread used CPU since the last one, so blocked time reads as waiting on every platform. The browser build has no profiler. Off, it costs nothing; on, a sample is one flag test at the next loop or call.

Development vs Production

```
# Development - all debug features
flarisvm script.fls --verbose --time

# Production - no debug info (the default), compiled with --small
flarisvm --exec app.flx
```

Best Practices

- Run with `-g` during development so stack traces carry line numbers
- Use meaningful breakpoint codes - treat them as documentation
- Combine `--verbose` + `breakpoint` for step-through investigation
- Use `Debug.Assert` liberally in test code
- Use `Debug.GuardAddress` for hard-to-find memory corruption

> See **Reference R11** for the complete Debug module API and opcode reference.

---

13. 10-Minute Tours

These tours help you get productive quickly if you're coming from another language.

Tour: Coming from C#

Mental Model

> **Flaris ≈ C# syntax + scripting flexibility + VM predictability**

What feels familiar: classes, methods, `try/catch`, loops, `async/await`
What is different: no static typing, no GC, fibers instead of Tasks, `nil` instead of null references

Side-by-Side Basics

Variables:

C#	Flaris
`int x = 10;`	`let x = 10;`
`var x = 10;`	`var x = 10;`
`const int MAX = 100;`	`const MAX = 100;`
`x++`	`x += 1;`

Functions:

C#:

```
int Add(int a, int b) { return a + b; }
Func<int, int> f = x => x * 2;
```

Flaris:

```
fn add(a: int, b: int): int { return a + b; }
var f = fn(x) => x * 2;
```

Arrays vs `List<T>` + LINQ:

C#:

```
var xs = new List<int> { 1, 2, 3 };
var doubled = xs.Select(x => x * 2).ToList();
```

Flaris:

```
var xs = [1, 2, 3];
var doubled = Array.Select(xs, fn(x) => x * 2);
```

`null` vs `nil`:

C# (dangerous):

```
User u = null;
Console.WriteLine(u.Name);  // NullReferenceException!
```

Flaris (safe):

```
var u = nil;
Console.WriteLine(u.name);  // nil (no exception)
```

All nil access is safe by default. Use `guard()` when you want to assert non-nil.

Classes:

C#:

```
class Point {
    public int X, Y;
    public Point(int x, int y) { X = x; Y = y; }
    public void Move(int dx, int dy) { X += dx; Y += dy; }
    public static Point Origin() => new Point(0, 0);
}
```

Flaris:

```
class Point {
    var x:int = 0;
    var y:int = 0;

    fn Constructor(x, y) {
        this.x = x;
        this.y = y;
    }
    fn move(dx, dy) {
        this.x += dx;
        this.y += dy;
    }
    fn static origin() {
        return new Point(0, 0);
    }
}
```

Key differences: no access modifiers, dynamic dispatch by default, no type declarations.

Async/Await:

C#:

```
async Task<int> Compute() { return 42; }
var v = await Compute();
```

Flaris:

```
fn async compute() { return 42; }
var v = await compute();  // compute() returns a fiber
```

Mental model: a **fiber** ≈ a `Task`, but cooperatively scheduled. No OS threads, no preemption, no race conditions.

Errors:

```
// Nearly identical to C#
try {
    doWork();
} catch (e) {
    Console.WriteLine(e.Error);
} finally {
    cleanup();
}
```

Mental Summary for C# Developers

- ✔ Think like C# for **structure**
- ✔ Think like a scripting language for **flexibility**
- ✔ Think like a VM for **performance**
- ✖ Don't rely on static typing
- ✖ Don't rely on GC behavior - use reference counting patterns

---

Tour: Coming from JavaScript

Mental Model

> **Flaris ≈ JavaScript syntax + explicit semantics + VM predictability**

What feels familiar: dynamic typing, objects and arrays, closures, `async/await`
What is different: no `undefined`, safe `nil`, explicit globals, fibers not event loop

Concept	JavaScript	Flaris
`undefined`	Yes	No - just `nil`
null	Dangerous	Safe `nil`
Globals	Implicit	Explicit `let` at top level
Prototypes	Implicit	Classes
Event loop	Mandatory	Fiber scheduler
Implicit coercion	Yes	No

Side-by-Side

Variables:

```
// JavaScript
let x = 10;
x++;

// Flaris
```

```
let x = 10;
x += 1;     // or: x++ / ++x (a statement, not an expression)
```

Arrays:

```
// JavaScript
xs.map(x => x * x);

// Flaris
Array.Select(xs, fn(x) => x * x);
```

Objects:

```
// JavaScript
let user = { name: "Ada" };
console.log(user.age);  // undefined

// Flaris
let user = { name: "Ada" };
Console.WriteLine(user.age);  // nil
```

Functions:

```
// JavaScript
const f = x => x * 2;

// Flaris
let f = fn(x) => x * 2;
```

Async/Await:

```
// JavaScript (event loop)
async function f() { return 42; }
let v = await f();

// Flaris (fiber scheduler)
fn async f() { return 42; }
let v = await f();
```

Classes:

```
// JavaScript
class Point {
    constructor(x, y) { this.x = x; this.y = y; }
}
```

```
// Flaris (instance fields must be declared)
class Point {
    let x = 0; let y = 0;
    fn Constructor(x, y) { this.x = x; this.y = y; }
}
```

Mental Summary for JavaScript Developers

- ✔ Familiar syntax
- ✔ Predictable semantics - no coercion surprises
- ✔ No event-loop surprises
- ✖ No `undefined`
- ✖ No implicit coercion

Tour: Coming from Rust

Mental Model

> **Rust's safety goals applied to a dynamic, embeddable VM**

What feels familiar: predictable execution, no undefined behavior, explicit control flow, strong safety defaults
What is different: dynamic typing, reference counting (not borrowing), runtime errors (not `Result<T,E>`)

Concept	Rust	Flaris
Typing	Static	Dynamic
Memory	Ownership + borrowing	Reference counting
Nulls	`Option<T>`	Safe `nil`
Errors	`Result<T, E>`	Runtime exceptions
Concurrency	Threads + async	Fibers
Abstraction cost	Zero-cost	Explicit, visible

Side-by-Side

Variables:

```
let mut x = 10;
x += 1;
const MAX: i32 = 100;
```

```
let x = 10;
x += 1;     // or: x++ / ++x (a statement, not an expression)
const MAX = 100;
// All variables are mutable unless const. No type annotations required.
```

Functions:

```
fn add(a: i32, b: i32) -> i32 { a + b }
```

```
fn add(a, b) { return a + b; }
// or with optional annotations:
fn add(a: int, b: int): int { return a + b; }
```

Arrays vs `Vec<T>`:

```
let xs = vec![1, 2, 3];
let sum: i32 = xs.iter().sum();
```

```
let xs = [1, 2, 3];
let sum = Array.Reduce(xs, fn(acc, x) => acc + x, 0);
```

`Option<T>` vs `nil`:

```
let u: Option<User> = None;
// u.name -> compile error
```

```
let u = nil;
Console.WriteLine(u.name);  // nil - safe, no panic
```

Error handling:

```
fn div(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 { Err("div by zero".into()) }
    else { Ok(a / b) }
}
```

```
fn div(a, b) {
    if (b == 0) throw(100, "div by zero");
    return a / b;
}
try { div(10, 0); } catch (e) { Console.WriteLine(e.Error); }
// Rust: explicit propagation. Flaris: explicit error boundaries.
```

Concurrency:

```
std::thread::spawn(|| { /* work */ });
```

```
let f = Fiber.New(fn() { /* work */ });
Fiber.Resume(f);
// Or async:
```

```
fn async task() { return 42; }
let v = await task();
```

Mental model: no data races by default, cooperative scheduling.

Mental Summary for Rust Developers

- ✔ Predictable execution
- ✔ No undefined behavior
- ✔ Explicit performance costs
- ✔ Strong safety defaults
- ✖ No compile-time type guarantees
- ✖ No zero-cost abstractions
- ✖ Errors are runtime, not typed

Think of Flaris as **a safe, dynamic VM - not a systems language**.

---

14. Package Manager (flarispm)

`flarispm` is the official Flaris package manager - written entirely in Flaris itself. It installs third-party libraries from any public or private git repository (or from a direct `.flx` URL), compiles them to `.flx` bytecode, and places them in the per-user library directory (`~/.flaris/libs`, or `$FLARIS_LIBS`) where `flarisvm` resolves them automatically - no `--libs` flag needed.

Installation

`flarispm` ships as two files alongside `flarisvm`:

File	Purpose
`flarispm.flx`	Compiled package manager bytecode
`flarispm`	Shell wrapper - runs `flarisvm --exec flarispm.flx "$@"`

Copy both to the same directory as `flarisvm` (e.g. `/usr/local/bin/`). The wrapper finds `flarispm.flx` next to itself automatically.

```
# Verify installation
flarispm version
# flarispm 0.5.0
```

`flarispm.flx` itself is signed with the official flaris-lang.org key - verify any copy with `flarisvm --sig-info flarispm.flx`.

Prerequisites: `git` in `PATH` for git-based packages; `curl` (ships with macOS, most Linux distros, and Windows 10+) or `wget` for direct `.flx` downloads.

Quick Start

```
# 1. Create a manifest in your project directory
flarispm init

# 2. Add a package from GitHub (or any git host)
flarispm add https://github.com/user/mylib

# 3. Run your program — installed packages resolve automatically
flarisvm myapp.fls
```

That's the entire workflow. `flarispm add` clones, compiles, verifies, and registers the package in one step.

The `flaris.json` Manifest

Every project managed by `flarispm` has a `flaris.json` at its root. `flarispm init` creates one:

```
{
  "name": "my-app",
  "version": "0.1.0",
  "description": "My Flaris application",
  "author": "Your Name",
  "dependencies": {
    "https://github.com/user/mylib": "main",
    "https://github.com/user/other": "v1.2.0",
    "https://example.com/tool.flx": "direct"
  }
}
```

Dependency keys are git clone URLs or direct `.flx` download URLs. **Values** are the branch name, tag, or `"main"` for the default branch (`"direct"` for `.flx` URLs).

This file is yours to edit: it says what you asked for. Nothing in it identifies a specific build - `"main"` names different code tomorrow than it does today.

URLs are restricted to `https://` (plus `ssh://` / `git@` for git) - plain-http and exotic git transports are refused, both from the command line and from a cloned manifest.

The `flarispm.lock` Lockfile

Beside the manifest, `flarispm` maintains `flarispm.lock`. Where the manifest records what you *asked for*, the lock records what was actually *resolved*:

```
{
  "lockfileVersion": 1,
  "packages": {
    "https://github.com/user/mylib": {
```

```
      "requested": "main",
      "resolved":  "3216f0772242d1429b57e614c0fcd85117db2389",
      "direct":    true,
      "modules":   [ "MyLib.flx" ],
      "integrity": { "MyLib.fls": "sha256:..." }
    },
    "https://example.com/tool.flx": {
      "requested": "direct",
      "resolved":  "direct",
      "direct":    true,
      "modules":   [ "tool.flx" ],
      "integrity": { "tool.flx": "sha256:..." },
      "publisher": { "key": "ed25519:...", "signer": "example.com" }
    }
  }
}
```

Field	Meaning
`requested`	the ref from `flaris.json` - the branch or tag you asked for
`resolved`	the exact commit that ref pointed at when it was pinned (`"direct"` for `.flx`)
`direct`	`true` when you asked for this package; `false` when it came in as a dependency
`modules`	the `.flx` filenames this package installs - the basis of collision detection
`integrity`	source hashes for a git package, exact bytes for a `.flx`
`publisher`	the pinned Ed25519 key, for signed `.flx` packages

Commit pinning is what makes a build reproducible. `"main"` is a moving target; `resolved` is not. `flarispm install` checks out the recorded commit even when the branch has since moved on, so a colleague cloning your project builds the same code you did. Only `add` and `update` move a pin.

Commit both files. The manifest alone reproduces an approximation; the lock reproduces the build.

A project created before the lockfile existed keeps its pins: the first `add`, `install` or `update` moves `integrity` and `publishers` out of `flaris.json` into `flarispm.lock` and reports that it did so.

Commands

`flarispm init`

Creates `flaris.json` in the current directory. Does nothing if one already exists.

```
flarispm init
# [✓] Created flaris.json
```

`flarispm add <url> [version] [--key <pubkey>]`

Clones the package, compiles all `.fls` files in its root, installs the resulting `.flx` files into `~/.flaris/libs`, records their SHA-256 pins, and registers the dependency in `flaris.json`. For a direct `.flx` URL it

downloads instead of cloning, and additionally pins the publisher's Ed25519 key if the file is signed.

```
# Default branch
flarispm add https://github.com/user/mylib

# Specific tag
flarispm add https://github.com/user/mylib v2.0.0

# Private / self-hosted git
flarispm add https://git.mycompany.com/internal/utils

# Pre-compiled, signed .flx — verify the publisher key while adding
flarispm add https://example.com/tool.flx --key <ed25519-pubkey-hex>
```

`add` is the only command allowed to *move* an existing integrity pin - use it deliberately when you intend to accept a new release of a direct `.flx` package.

After `add`, import the package by the name of its exported `.fls` file:

```
import { MyLib } from library("MyLib", "1.0");
```

Source is cached in `~/.flaris/libs/.src/` - re-running `add` on an already-installed package skips the clone and only recompiles.

`flarispm install`

Reads `flaris.json` and installs every listed dependency. Use this after cloning a project someone else created:

```
git clone https://github.com/user/my-project
cd my-project
flarispm install
flarisvm main.fls
```

`install` reproduces `flarispm.lock` exactly. For a git package it checks out the recorded commit - not the branch tip - and refuses if that commit no longer exists upstream. Every download and every pinned source is verified against its `sha256:` pin (and, for signed `.flx` packages, the pinned publisher key) **before** anything is moved into `~/.flaris/libs`. A mismatch is a hard refusal: `install` never re-pins. If upstream legitimately released something new, accept it explicitly with `flarispm add` (direct `.flx`) or `flarispm update` (git packages).

Packages the manifest does not list but a dependency requires are installed too, in dependency order - a library's own `flaris.json` declares its dependencies exactly as a project does, and they are on disk before anything that imports them compiles. Cycles terminate, and a chain deeper than 16 is refused.

`flarispm list`

Shows all registered packages and whether their source is present locally:

```
flarispm list
# Packages in flaris.json:
#   mylib @ v2.0.0  [installed]
#     https://github.com/user/mylib
#   utils @ main  [installed]
#     https://github.com/user/utils
```

`flarispm update [url]`

Pulls the latest commits for all packages (or one specific package), recompiles, and moves the lock to what it fetched - the new commit, the new source hashes, and for a signed `.flx` the publisher key it arrived with:

```
flarispm update                              # update everything
flarispm update https://github.com/user/mylib  # update one package
```

> **Note:** If the package is pinned to a tag (`"v1.2.0"`), `git pull` will have nothing new to fetch. To upgrade to a newer tag, use `flarispm remove` then `flarispm add` with the new version.

`flarispm verify`

Checks what is installed against `flarispm.lock` and changes nothing. Exits non-zero on the first failure, so CI can gate on it:

```
flarispm verify || exit 1
```

It confirms, per package, that:

- every module the lock says the package owns is present in the library directory;
- pinned bytes still hash to their pin (`.flx`), and pinned sources still hash to theirs (git);
- the installed `.flx` of a source package is still what its pinned `.fls` compiles to - a source pin alone would not notice a swapped artifact;
- a git package's checkout still sits on the locked commit;
- a signed package still carries a valid signature from the pinned publisher key.

`flarispm prune`

Removes packages nothing references any more. Reachability is recomputed from `flaris.json` outward through each installed package's own dependencies, so removing a library also retires whatever it alone pulled in:

```
flarispm remove https://github.com/user/mylib
# [·] 2 package(s) now unreferenced: run 'flarispm prune' to remove.
flarispm prune
```

`flarispm remove <url>`

Removes the compiled `.flx` files produced by a package and unregisters it - including its integrity and publisher pins - from `flaris.json`. The cloned source in `~/.flaris/libs/.src/` is kept as a cache - a subsequent `add` reuses it and skips the network round-trip.

```
flarispm remove https://github.com/user/mylib
# [·] Deleted MyLib.flx
# [✓] Removed mylib from flaris.json
```

Using Installed Packages

Installed packages live in `~/.flaris/libs` (override with `$FLARIS_LIBS`) as `.flx` files - the same per-user directory `flarisvm` searches automatically, so no flag is needed.

Inside your code, import exactly as you would any stdlib module - the name comes from the `.fls` filename in the package:

```
// Package installed from https://github.com/stefansolid/flaris-test-package
// which contains Greet.fls → compiled to ~/.flaris/libs/Greet.flx

import { Greet } from library("Greet", "1.0");

fn Main() {
    Greet.Hello("Coder");    // Hello, Coder!
    Greet.Shout("it works");  // IT WORKS!!!
}
```

Run:

```
flarisvm myapp.fls
```

Directory Layout

The manifest stays in your project; the installed packages are per-user:

```
my-project/
├── flaris.json              ← manifest + pins (commit this)
└── main.fls
```

```
~/.flaris/libs/              ← or $FLARIS_LIBS
├── MyLib.flx                ← installed packages (build artifacts)
├── Utils.flx
├── .staging/                ← downloads/compiles awaiting verification
│                              (never resolvable by the module loader)
└── .src/                    ← cloned git sources (cache)
    ├── github.com_user_mylib/
    └── github.com_user_utils/
```

Nothing package-related needs a `.gitignore` entry - only `flaris.json` lives in the project, and it *should* be committed. Anyone who checks out your project runs `flarispm install` to reproduce the exact pinned dependencies.

Creating a Package

Any git repository containing `.fls` files is a valid Flaris package. The minimal structure:

```
my-package/
├── flaris.json    ← required: declares name and version
└── MyLib.fls      ← one or more library files
```

`flaris.json` for a package:

```
{
  "name": "MyLib",
  "version": "1.0.0",
  "description": "What this library does",
  "author": "Your Name",
  "homepage": "https://github.com/user/my-package"
}
```

`MyLib.fls` - define and export your API:

```
// MyLib.fls - Example library
//
// Usage:
//   import { MyLib } from library("MyLib", "1.0");
//   MyLib.Greet("world");

class MyLib {

    fn static Greet(name?: string) {
        let n = name != nil ? name : "World";
        Console.WriteLine("Hello, " + n + "!");
    }
```

```
    fn static Version() : string {
        return "1.0.0";
    }
}

export { MyLib };
```

Publish by pushing to any git host - no registry needed. Users install with:

```
flarispm add https://github.com/you/my-package
```

Version Tags

Tag your releases so users can pin to a stable version:

```
git tag v1.0.0
git push origin v1.0.0
```

Users can then install the exact release:

```
flarispm add https://github.com/you/my-package v1.0.0
```

Version Matching

When `flarispm` compiles a package it embeds the `version` field from the package's `flaris.json` into the bytecode using `--min-version=<ver>`. The `library()` call in your code specifies a version requirement that must match:

```
// Matches any 1.x release
import { MyLib } from library("MyLib", "1.0");

// Exact match
import { MyLib } from library("MyLib", "1.2.3");

// Range syntax
import { MyLib } from library("MyLib", ">=1.0 <2.0");
```

If the version in the bytecode does not satisfy the requirement, the VM halts before execution. This prevents accidentally running an incompatible version of a library.

Security - Integrity and Publisher Pinning

`flarispm` enforces four layers, all recorded in `flarispm.lock` and all verified while a file is still staged - a package that fails any check never reaches the import path:

1. **Integrity pinning (all packages).** Every package gets a `sha256:` pin under `integrity` (trust-on-first-use at `add` time), and on every later `install` the pin must still match or the package is refused. What is pinned differs by package kind: a direct `.flx` download is pinned by its **exact bytes**, while a git package is pinned by the **`.fls` source** it was compiled from, plus the **exact commit** under `resolved`. `install` can never move a pin; only an explicit `add` or `update` accepts a new value.
2. **Publisher continuity (signed `.flx` packages).** When a signed `.flx` is added, the publisher's Ed25519 key is pinned under `publishers`. Every later download must be signed by the *same* key - a tampered file, a stripped signature, or a silently swapped key all block the install. Accept an intentional key rotation by re-adding with `--key <new-pubkey>`.
3. **Publisher verification at add (`--key`).** `flarispm add <url> --key <pubkey>` refuses the package unless it is signed by exactly that key - use it when the publisher advertises their key out-of-band (README, website).
4. **Module ownership.** Every package installs into one flat directory, so two packages that both ship `Util.fls` would race to own `Util.flx` and the loser's importers would silently bind to the winner's code. The lock records which package owns each installed filename; a second claimant is refused outright, naming both packages. This matters most for transitive dependencies, where you never chose the colliding package yourself.

Git/source packages are compiled locally and left unsigned on purpose: a signature applied at compile time would attest to the *installer*, not the publisher. They are trusted via the git ref plus the sha256 pin of their `.fls` source instead.

The source is what gets pinned, not the compiled `.flx`, for two reasons. A compiled artifact embeds debug info including the absolute source path, so its hash differs on every machine - pinning it would make a committed `flaris.json` reproducible only for its author. And the source hash is the more meaningful attestation anyway: it detects upstream source that changed since you pinned it, which is the actual threat. A mismatch means the publisher moved the code under a ref you pinned - review it, then `flarispm update <url>` to accept and re-pin.

Signature inspection is delegated to the VM: `flarisvm --sig-info <file.flx>` prints `unsigned` or `signed|<trusted|valid|invalid>|<pubkey>|<signer>`.

VM-side Fingerprint Pinning

For security-sensitive dependencies you can additionally pin the fingerprint at the *import site*, enforced by the VM itself on every run. Get the fingerprint after installing:

```
flarisvm --sha256 ~/.flaris/libs/MyLib.flx
# Fingerprint(SHA256): [a3f1...64hexchars...]
```

Then lock the import to that exact binary:

```
import { MyLib } from library("MyLib", "1.0", "a3f1...64hexchars...");
```

If the bytecode does not match - even if the version is correct - the VM halts immediately. Use this for libraries that handle credentials, crypto, or authentication.

The fingerprint is the whole-file SHA-256 (`flarisvm --sha256 == sha256sum`), recomputed from the bytes actually loaded - there is no self-certifying hash inside the file. A `sha256:` prefix (the form flarispm stores under `integrity` in `flaris.json`) is accepted, so you can paste that value directly. The pin is always enforced and is not affected by `--no-verify` .

Publisher signatures

Where the fingerprint pins the *exact bytes*, an Ed25519 **publisher signature** pins the *author* - and survives version bumps. When you add a signed `.flx` package, flarispm records the publisher's public key under `publishers` in `flaris.json` , and every later `install` / `update` must be signed by that same key, or it is refused (catching a tampered file or a silently swapped key):

```
# Pin the publisher to the key you expect (cross-checked before recording)
flarispm add https://example.com/MyLib.flx --key <ed25519-pubkey-hex>
```

Git/source packages are compiled locally and left unsigned on purpose - a signature applied at install time would attest to *you*, not the publisher - so they rely on the git ref plus fingerprint instead. To enforce signatures at runtime, trust the publisher's key (`flarisvm --import-key <name> <pubkey>`) and run with `--require-signed` .

A `.flx` is executable code, not a sandbox. It can touch the filesystem, spawn processes, and (with `--unsafe`) call FFI and raw memory - so only run bytecode you trust, and pin third-party libraries to a known hash. Note that the JIT widens the trust surface: JIT-compiled code omits the interpreter's runtime type checks, so a `.flx` is JIT-compiled only when it is signed by a trusted key - anything else runs in the interpreter. See Reference R1 "Security model".

Environment Variables

Variable	Effect
`FLARISVM`	Override the path to `flarisvm` used for compiling packages. Useful when multiple versions are installed.

---

*End of Flaris Guide. For complete API reference, type system details, stdlib documentation, performance tuning, and VM internals, see the **Flaris Reference**.*